SPOT — Guia de estudo e defesa do TCC

Da pesquisa de campo à arquitetura do sistema

Colégio Dr. Clóvis Bevilacqua — Curso de Informática

Alexander Molany Longo; Lucas Rodrigues Soares; Pedro Galvão de Matos; Yasmin Regina de Souza

Orientador indicado na monografia: João Carlos de Souza

Santo André — 2026

Data de corte técnico: 26 de outubro de 2026.

1. Como utilizar este material

A apresentação tem 15 slides e um roteiro estimado de 14min10s, deixando 50 segundos de margem até o limite de 15 minutos. O texto de fala está nas notas do PowerPoint e na seção 16 deste guia. Não é necessário ler este documento inteiro durante a apresentação: ele funciona como material de estudo, rastreabilidade e consulta para perguntas.

As fontes primárias fornecidas são o capítulo Capitulo_Levantamento_Pesquisa_ABNT.docx, a Monografia_SPOT_documentacao_integrada.docx e a implementação local do Spot. A monografia contém dois trabalhos de pesquisa diferentes: o levantamento com usuários e o estudo de sistemas do mercado, incluindo tecnologias de IA. Ambos são articulados aqui com a implementação. Os arquivos originais não foram alterados. A revisão desta entrega completa os itens funcionais parciais da tabela e retira IA da lista de implementação, preservando os dados brutos da pesquisa.

Este guia cobre o código e os documentos disponíveis nesta sessão e no workspace. Não pressupõe acesso a outras conversas do Codex. As imagens do sistema são capturas da interface real do build local, com dados demonstrativos e respostas de API simuladas. Nos slides, o perfil etário usa gráfico de linhas e a necessidade de software usa gráfico de setores. As barras também exibem contagem e porcentagem sobre a base declarada de 94. Os gráficos da pesquisa foram reconstruídos a partir do texto e das imagens incorporadas ao DOCX, sem acesso à planilha individual de respostas.

2. Tese central e problema atendido

O SPOT — Sistemas de Projetos, Organização e Tarefas — foi desenvolvido para responder às necessidades da Computécnica. A monografia relata visual antigo, dificuldade de integração entre telas, responsabilidades pouco delimitadas, restrições insuficientes de

acesso, despesas adicionais desconectadas do planejamento e dependência de licença do sistema anterior.

A proposta combina três perspectivas: organização do trabalho, governança do acesso e acompanhamento financeiro gerencial. O fluxo central é cadastrar o contrato/projeto, planejar custos e horas, atribuir equipe, registrar atividades e despesas e comparar o resultado com a estimativa. Assim, a contribuição do trabalho vai além de redesenhar telas: transforma requisitos de uma operação real em regras verificáveis e dados relacionados.

Frase de abertura sugerida: “O Spot nasceu para que a Computécnica acompanhe não apenas o andamento de um projeto, mas também quem pode agir sobre ele e quanto sua execução está custando.”

3. Pesquisa de campo: metodologia e qualidade

O documento caracteriza a pesquisa como aplicada, descritiva e quali-quantitativa, realizada por Google Forms. Declara 15 perguntas fechadas e uma aberta, coleta entre 18/03/2026 e 24/08/2026 e 94 respostas válidas. A amostra é de conveniência e não probabilística. Isso permite descrever padrões entre participantes e orientar requisitos; não permite estimar o comportamento de todas as empresas, calcular representatividade populacional ou atribuir causalidade.

Pontos fortes: combinação de contagens e sugestões abertas; 94 participantes; diversidade de faixas etárias; 88 pessoas empregadas; presença de pessoas que não são especialistas em gestão de projetos; levantamento de dores, ferramentas, usabilidade e necessidades financeiras. O emprego de perguntas abertas ajuda a transformar uma preferência genérica em requisitos concretos.

A participação de não especialistas é útil para discutir adoção cotidiana, mas não prova validade externa nem substitui validação com gestores e analistas da Computécnica. As percepções também não são medidas diretas de produtividade, retorno financeiro ou sucesso de implantação.

3.1 Cuidados com os números

Várias figuras exibem totais entre 89 e 92, embora o texto declare 94 participantes. Neste material, percentuais principais usam 94 como denominador, conforme o capítulo. As diferenças são identificadas como “diferença até 94 não discriminada no documento”. Não é possível afirmar se resultam de não resposta, filtro, limpeza ou erro sem a base original.

A questão de ferramentas permite menções não exclusivas; suas contagens não devem somar 94. O capítulo apresenta 13 gráficos, a informação de emprego e a síntese aberta; não permite reconstruir separadamente todas as 15 perguntas fechadas declaradas. Não foi inventada uma pergunta ausente.

A tabela qualitativa soma 49 frequências, mas o texto informa 57 comentários. Sem a codificação individual não se deve criar uma oitava categoria ou assumir exatamente oito

respostas não classificadas: a diferença de oito unidades é uma inconsistência documental a reconciliar, e categorias também poderiam se sobrepor.

O texto afirma que existe maioria favorável a uma solução dedicada entre quem já possui ferramenta. Não há tabela cruzada nem base individual para conferir essa conclusão; por isso, ela não foi usada como evidência quantitativa nos slides.

4. Todos os resultados recuperáveis e sua relação com o Spot

As contagens abaixo transcrevem as figuras e o texto fornecidos. A pasta assets/graficos contém os 13 gráficos reconstituídos e o gráfico temático; dados_pesquisa.json e dados_pesquisa.csv permitem conferir os valores.

4.1 Perfil: idade, emprego e atuação

Faixa etária, figura 1: 15–25 anos, 28; 25–35, 22; 35–45, 19; 45–55, 15; acima de 55, 10. Total 94. Respectivamente: 29,8%, 23,4%, 20,2%, 16,0% e 10,6%. Empregados: 88/94, ou 93,6%; o documento não discrimina aqui os demais estados de emprego.

Atuação em projetos, figura 2: não, 65; parcialmente, 20; totalmente, 4; sim, 3. Soma 92; diferença de 2 até 94. O capítulo agrega as duas últimas categorias em 7, mas o gráfico mantém as alternativas separadas. 65/94 = 69,1% não atuam diretamente na área.

Relação com a implementação: quadro/lista/calendário, descrições claras, tutorial guiado, três idiomas e navegação por áreas foram adotados para facilitar o uso. A pesquisa justifica a prioridade de usabilidade; ainda é necessário medir a facilidade real com participantes, tarefas e critérios definidos.

4.2 Ferramentas utilizadas

Figura 3: Office 365, 52 menções (55,3% de 94); software próprio, 26; Office 2021, 19; nenhum software, 17 (18,1%); Bloco de Notas, 16; Notion, 4. Total de 134 menções, compatível com contagens não exclusivas.

Relação com o Spot: login Microsoft, calendário Outlook, criação de reuniões Teams e diretório Microsoft Graph respondem à presença do ecossistema Microsoft. Também há login e agenda Google. Não há importação genérica de todos os aplicativos citados. O menu de 24 aplicativos abre serviços externos; não equivale a 24 integrações de dados.

4.3 Software específico de projetos

Figura 4: não possui, 33 (35,1%); possui, 32 (34,0%); possui, mas projetos não são seu foco, 27 (28,7%). Soma 92; diferença 2. As categorias “não” e “sem foco” somam 60/94 = 63,8%. Essa soma é derivada para este material e não deve ser confundida com os outros 63,8% da questão de organização.

Relação com o Spot: aplicação dedicada à operação de projetos, conectando clientes, responsáveis, atividades, horas e resultado. A existência de uma oportunidade nessa amostra não comprova disposição de compra nem participação de mercado.

4.4 Satisfação e usabilidade atuais

Figura 5: “sim, porém poderia haver melhorias”, 30; “não temos”, 29; “sim, ele é competente no que precisamos”, 23; “não, no dia a dia nem é utilizado”, 7. Soma 89; diferença 5. Esses dados mostram convivência de satisfação e oportunidades de melhoria. Não se deve dizer que a maioria está insatisfeita.

Figura 6: “não possuímos um software para isso”, 29; “sim”, 28; “mais ou menos”, 25; “não”, 4; “absolutamente”, 4. Soma 90; diferença 4. A presença de 25 respostas intermediárias sugere investigar dificuldades de interação. As 29 pessoas sem software não avaliam propriamente uma interface existente.

Relação com o Spot: identidade visual, temas claro/escuro, filtros, diferentes visualizações e tutorial. São respostas de design e implementação, não resultados de um teste comparativo de usabilidade. Não foi aplicado SUS ou ensaio controlado nessa execução.

4.5 Projetos internos e externos

Figura 7, externos: sim, 42; não, 21; quase nunca, 18; absolutamente, 9. Soma 90; diferença 4. Concordância direta: 51/94 = 54,3%.

Figura 8, internos: sim, 57; absolutamente, 16; não, 9; quase nunca, 7. Soma 89; diferença 5. Concordância direta: 73/94 = 77,7%.

Relação com o Spot: projetos com ou sem cliente, equipes, convites e vínculos; a API prevê modalidades de participação e referências externas. Há base de modelagem, mas a interface não demonstra dois workflows completos e separados de ponta a ponta para interno e externo. Classificação desse requisito: parcial.

4.6 Organização e alinhamento

Figura 9, dificuldade de organização: sim, 49; não, 16; talvez, 13; absolutamente, 11. Soma 89; diferença 5. Concordância: 60/94 = 63,8%.

Relação com o Spot: Home, cadastro e acompanhamento de projetos, tarefas em quadro/lista/calendário, filtros por ano/cliente/projeto/status, prioridades, prazos, responsáveis e histórico. A implementação responde à dor, mas a pesquisa não mede se o Spot já a reduziu.

Figura 10, dificuldade de alinhamento na liderança: não, 31; sim, 31; talvez, 20; absolutamente, 8. Soma 90; diferença 4. Concordância: 39/94 = 41,5%. Não foi demonstrado que os demais são todos líderes; não atribuir a dificuldade a uma causa específica sem dados.

Relação com o Spot: equipe por projeto, responsável e coparticipantes na tarefa, funções, histórico e regras de acesso. Não há chat interno completo nem prova de redução de reuniões ou conflitos.

4.7 Organização e faturamento

Figura 11: sim, 57; absolutamente, 26; talvez, 4; não, 4. Soma 91; diferença 3. Concordância: 83/94 = 88,3%.

Relação com o Spot: custos por analista e atividade, tarifas normais/extras, despesas aprovadas, impostos, comissão e relatórios. Esse é o achado quantitativo mais forte para justificar a união entre organização e financeiro. A pergunta mede percepção; não demonstra aumento de faturamento causado pelo sistema.

4.8 Custos adicionais

Figura 12: não, 33; talvez, 26; sim, 21; absolutamente, 9. Soma 89; diferença 5. Dificuldade direta: 30/94 = 31,9%. Não é a dor majoritária da amostra; continua relevante pela demanda específica da Computécnica e pelo grupo que a relata.

Relação com o Spot: tipos de despesas, lançamento, analista associado quando autorizado, revisão, aprovação/rejeição e incorporação apenas das aprovadas no resultado. Viagem ou hospedagem podem ser organizadas como tipos de despesa; não existe integração automática com reservas ou cartões corporativos.

4.9 Necessidade de solução dedicada

Figura 13: talvez, 33; sim, 32; não, 14; absolutamente, 11. Soma 90; diferença 4. Apoio direto: 43/94 = 45,7%. Apoio mais abertura: 76/94 = 80,9%, arredondado no documento para aproximadamente 81%. “Talvez” não é intenção de compra nem aprovação definitiva. Não dizer que 81% comprariam o Spot.

Relação com o Spot: oferta de uma solução dedicada e adaptável ao contexto. A próxima etapa seria validação com usuários-alvo, piloto e avaliação de esforço de implantação.

4.10 Sugestões abertas

O documento declara 57 comentários, equivalentes a 60,6% dos 94 participantes. O recorte funcional adotado para a apresentação destaca:

Tema	Frequência publicada	Resposta no Spot	Situação
Interface simples	12	Tutorial, temas, navegação, visões de tarefas	Implementado; eficácia ainda não medida
Financeiro/custos	6	Horas, tarifas, despesas e planejado × realizado	Implementado, gerencial
Integrações/e-mail	6	OAuth, calendários paginados, Teams e e-mails de atividade em fila	Implementado; exige credenciais, SMTP e ativação
Dashboards	5	Portfólio, gráficos, CSV e atualização automática	Implementado; consulta periódica de 30 s

Tarefas/agenda/prazos	5	Quadro/lista/calendário, filtros, agenda e lembretes automáticos	Implementado; canais, silêncio e resumos respeitados
Sem sugestão específica	12	Sem requisito específico derivado	Categoria da pesquisa

As descrições são sínteses temáticas do documento, não falas literais dos participantes. A lista funcional seleciona os temas ligados às entregas; a codificação original completa continua nos dados JSON/CSV e no gráfico de apoio, sem redistribuir frequências. O pedido de indicadores foi atendido com atualização automática a cada 30 segundos nas telas visíveis, além de atualização após mutações e entre abas. É consulta periódica, não streaming ou WebSocket.

5. Pesquisa de sistemas: o que aprender e como comparar

A monografia estuda Monday.com, Asana, Jira, Trello, ClickUp e Notion e inclui AWS. Também discute ClickUp Brain, Asana Intelligence, Monday AI, TaskRush/RushMind, Notion AI, Trello/Butler, Artia e Jestor na seção de IA e planos.

Seu ponto forte é ampliar as referências de interação e escopo: organização visual, tarefas, colaboração, indicadores, documentação, integrações e automação. O texto registra explicitamente Monday.com como referência visual. A comparação ajuda a decidir o que adaptar ao cliente, em vez de construir funcionalidades sem referência.

Seu limite é não apresentar um protocolo de testes comparativos com tarefas iguais, tempo de execução, amostra, pontuação e custo total padronizado. Os rótulos “melhor”, “pouco intuitivo” e “excelente custo-benefício” da monografia são avaliações exploratórias, não conclusões experimentais. Os preços e nomes de planos registrados são históricos e podem misturar unidades, cobrança anual/mensal e recursos adicionais; não foram reproduzidos como tabela vigente.

Para a apresentação, foram consultadas páginas oficiais em 26/10/2026. Elas confirmam os posicionamentos e evitam negar funções que os fornecedores oferecem. As fontes estão na seção 20. Não houve teste prático dessas plataformas nesta tarefa.

Sistema pesquisado	Referência e ponto forte	Posicionamento defensável do Spot
Monday.com	Gestão visual, dashboards e automações	Fluxo financeiro e governança configurados para a Computécnica; não exclusividade de dashboards
Asana	Coordenação de tarefas, projetos e acompanhamento	Foco no contrato, mão de obra, despesas e margem da operação local
Jira	Planejamento e acompanhamento de trabalho; forte referência ágil	Escopo centrado na prestação de serviços e financeiro gerencial; não afirmar que Jira atende somente programadores
Trello	Quadros/cartões e organização visual,	Regras de tarifas, impostos, aprovação

	com automações e extensões	e resultado já modeladas no próprio domínio do Spot
ClickUp	Centralização, tarefas, documentação, tempo e dashboards	Modelo menor e específico para o cliente; não afirmar que só o Spot centraliza ou controla horas
Notion	Documentação, bases configuráveis e projetos	Regras transacionais e financeiras explícitas, em vez de depender de uma montagem genérica de bases
TaskRush/RushMind	Horas, custos e margem, além de IA	Há sobreposição financeira real. Diferencial do Spot é a adequação e o controle de evolução local, não inventar rentabilidade por projeto
Artia	Gestão de projetos e IA para planejamento	Spot concentra o recorte contratual/operacional identificado no cliente; não oferece o mesmo planejamento assistido por IA
Jestor	Aplicações/processos configuráveis e automação com IA	Spot entrega um domínio específico em código próprio; não demonstra a mesma amplitude de plataforma configurável
AWS	Infraestrutura e serviços de nuvem	Pode hospedar uma solução como Spot; não é concorrente funcional direto de um gestor de tarefas

A alegação central é de adequação ao caso de uso. Código acessível e implantação própria permitem controlar mudanças e hospedagem. Isso não torna operação gratuita: existem despesas de servidor, suporte, manutenção, backups, administração e eventuais serviços externos. O repositório declara MIT no Composer e ISC no package.json; convém harmonizar a identificação da licença antes de uma distribuição comercial. Não há estudo de TCO ou contrato que sustente um número de economia.

6. Matriz de rastreabilidade: pesquisa → requisito → código

Evidência	Requisito implementado	Tela	Evidência técnica
60/94 com dificuldade de organização	Centralizar projetos e tarefas	Home, Projetos, Tarefas	HomeDashboardController; ProjectController; TaskV1Controller
39/94 com dificuldade de alinhamento	Tornar responsabilidades visíveis	Equipes e detalhes de tarefas	TeamController; ProjectMember; Task.participants
83/94 associam organização e faturamento	Relacionar execução ao resultado	Detalhes do projeto e Portfólio	ProjectFinancialSummary
30/94 com dificuldade de custos extras	Registrar e revisar despesas	Detalhes do projeto	ProjectExpenseController; ReviewProjectExpense
52 menções a Office 365	Conectar ecossistema corporativo	Agenda, Apps e administração	MicrosoftAuthController; CalendarController;

			DirectoryController
12 frequências de interface simples	Facilitar a aprendizagem	Interface geral e tutorial	App.jsx; TasksPage.jsx; SpotTutorial.jsx
5 frequências de dashboards	Dar visibilidade gerencial	Portfólios	PortfolioDashboard; ProjectAnalyticsController; project_analytics.py
Demandas de acesso da Computécnica	Autorizar por perfil/operação/campo	Administração e formulários	User.hasPermission; Gate; ProjectFields
Memorial: encerramento protegido	Finalizar/reabrir com justificativa	Projetos	FinalizeProject; ReopenProject
Memorial: retenção de logs	Auditoria com prazo configurável	Histórico e configurações	AuditRecorder; HistoryController; audit:prune
Fluxos internos/externos	Modelagem de participação e vínculo	Cadastro/API e equipes	SaveProjectRequest; ProjectMember — cobertura parcial

Demandas financeiras e de segurança também vieram do cliente e do memorial. Nem toda funcionalidade precisa ser atribuída ao survey. Estoque, por exemplo, é um módulo complementar e não o argumento principal da pesquisa de projetos.

7. Arquitetura e estrutura do repositório

A aplicação é um monólito web organizado por camadas: backend Laravel e frontend React no mesmo repositório e artefato de implantação, com separações de responsabilidade. Não são microserviços. Web, worker e scheduler podem rodar em processos/containers distintos, mas compartilham o domínio e a base da aplicação.

```
Spot/
├── app/
│   ├── Domain/                regras e ações de negócio
│   │   ├── Audit/             registro de eventos
│   │   ├── Finance/           cálculo financeiro
│   │   ├── Projects/          campos e ciclo de vida
│   │   ├── Tasks/             captura de tarifas
│   │   ├── WorkLogs/          apontamentos
│   │   ├── Expenses/          revisão de despesas
│   │   ├── Invitations/       convites e aceite
│   │   ├── Notifications/     eventos, preferências, lembretes e entregas
│   │   ├── Identity/          ativação
│   │   └── Inventory/         movimentação de estoque
│   ├── Http/Controllers/      entrada e respostas HTTP
│   ├── Http/Requests/         validação reutilizável
│   ├── Models/                entidades e relações Eloquent
│   ├── Jobs/                  processamento assíncrono
│   ├── Mail/                  mensagens de e-mail
│   └── Providers/             configuração de autorização/provedores
├── routes/                    web.php, api.php, console.php
├── resources/
│   ├── js/                    interface ativa React
│   ├── css/                   estilos da interface
│   └── views/                 HTML base Blade e modelo de e-mail
├── database/                  migrations, factories e seeders
├── config/                    autenticação, banco, sessões, filas etc.
├── scripts/                   análise Python, navegador e scheduler
├── tests/                     suíte PHP de unidade e funcionalidade
├── public/                    assets públicos e build
├── storage/                   arquivos privados, logs e dados de execução
├── bootstrap/                 inicialização Laravel
├── docker/                    configuração Caddy
├── docs/                      documentação e material do TCC
├── Dockerfile                 imagem com build frontend e execução PHP
└── docker-compose.yml         web, worker, scheduler e PostgreSQL
```

A entrada ativa está em resources/js/app.jsx, que monta App em #root. resources/views/app.blade.php entrega o HTML base e informações iniciais. vite.config.js aponta essa entrada. A existência de src/App.jsx não significa que seja a interface principal: use a configuração do build para identificar o código executado.

7.1 Comunicação entre as camadas

O navegador carrega o React e faz chamadas JSON à API /api/v1. O cliente compartilhado resources/js/api.js usa sessão do mesmo domínio. Para alterações, obtém o cookie CSRF e envia X-XSRF-TOKEN. O middleware exige autenticação e aplica limites de frequência. Controllers e Form Requests validam o conteúdo e o acesso. Ações de domínio executam regras e transações. Modelos Eloquent consultam e persistem no PostgreSQL.

Respostas de erro têm implicação de produto: 403 representa falta de permissão; 404 pode esconder um recurso fora do escopo; 409 sinaliza conflito de estado, como projeto finalizado; 422 indica validação ou regra de negócio. A interface apresenta mensagens de erro e só atualiza os registros após confirmação da API.

As rotas sem /v1 mantêm JWT como compatibilidade. A sessão principal não guarda token de autenticação em localStorage. Preferências visuais em localStorage não são o mecanismo de login.

8. Decisões arquiteturais e seus motivos

Os motivos a seguir são leitura técnica dos benefícios e compromissos das escolhas existentes, apoiada no código e na documentação; não são entrevistas com os autores sobre cada decisão histórica.

1. Laravel no backend. Reúne autenticação, validação, ORM, migrações, filas e testes. Reduz a necessidade de implementar infraestrutura comum do zero. O compromisso é depender de atualizações e convenções do framework.

2. React na interface. Permite estado local, formulários e alternância de visualizações sem recarregar a página inteira. O código usa navegação por estado em App.jsx; não foi identificado React Router como dependência.

3. Mesmo repositório e origem. Facilita implantação e sessão/CSRF. Não exige vários serviços de aplicação para um TCC com equipe pequena. O compromisso é coordenar releases frontend e backend.

4. Ações de domínio. Finalizar projeto, aceitar convite ou registrar movimento ficam em classes identificáveis. Isso ajuda a testar regras e evita duplicação. A separação é parcial: vários controllers ainda concentram regras.

5. PostgreSQL. Banco relacional único da versão atual, com transações, chaves, consultas e bloqueio de linhas. O ambiente de testes usa schema separado. A monografia que cita SQLite precisa de atualização.

6. Modelagem por entidades e associações. Projeto, membro, tarifa, tarefa, apontamento, despesa e imposto têm responsabilidades próprias. Relações preservam vínculos e evitam repetir cadastros em cada lançamento.

7. Duração em minutos. A interface recebe horas e minutos e converte para inteiro. Reduz ambiguidade de frações de hora; o valor monetário é calculado proporcionalmente por 60.

8. Tarifas com vigência e snapshot. O período encontra o preço correto e a fotografia preserva o histórico. Alterar uma tarefa em campos que afetam sua precificação pode disparar recaptura; isso é diferente de um reajuste futuro retroagir sozinho.

9. BigDecimal para finanças. Evita depender de ponto flutuante binário no cálculo principal. O arredondamento ocorre em centavos; agregações gráficas posteriores usam números Python/JavaScript.

10. Transação e lockForUpdate. Operações sensíveis releem e bloqueiam a linha antes de mudar estado. Estoque evita duas saídas concorrentes sobre o mesmo saldo. Há custo de contenção e necessidade de transações curtas.

11. Auditoria junto da mutação. Nos fluxos transacionais relevantes, a operação e seu histórico são confirmados ou revertidos juntos. Isso melhora rastreabilidade, mas não transforma a tabela em log inviolável.

12. Exclusão lógica e arquivamento. Projetos usam SoftDeletes; parâmetros são inativados para preservar referências. Não se aplica uniformemente a toda entidade: tarefas, por exemplo, têm exclusão efetiva no fluxo analisado.

13. Sanctum e sessão na SPA. Usa cookies e CSRF para o navegador do mesmo domínio; JWT fica no legado. Diminui a necessidade de manipular bearer tokens no frontend.

14. OAuth sem senha externa. Google/Microsoft autenticam e autorizam acesso às suas APIs. Tokens são criptografados no modelo; dependem de consentimento, expiração e renovação.

15. Python/Pandas separado por processo. Laravel filtra e autoriza os projetos e envia JSON. Python agrega sem conexão própria ao banco. Facilita análise, mas adiciona dependência e custo de inicialização por requisição; o timeout é de 30 segundos.

16. Fila para e-mails. Evita que SMTP prolongue a requisição de criação do convite. O worker revalida o link e tenta novamente em falhas. Depende de supervisão e não garante entrega exatamente uma vez.

17. Scheduler para manutenção. Retenção de auditoria, limpeza de mensagens e geração/entrega de notificações não dependem de uma aba aberta. Exige um agendador operacional e execução coordenada.

18. CSS e gráficos próprios. Favorece identidade visual e customização sem biblioteca de gráficos de alto nível. Aumenta responsabilidade por acessibilidade, contraste, escalas e manutenção.

19. Traduções e preferências no navegador. Três idiomas, temas e tutorial reduzem barreiras de adoção. Preferências não são sincronizadas automaticamente entre dispositivos.

20. Docker com processos separados. Reproduz dependências e separa web/fila/agendador/banco. Não fornece sozinho redundância, backup, monitoramento ou recuperação de desastre.

9. Modelo de dados e regras que a banca pode perguntar

Company, Person, User, Profile e Permission representam identidade, empresa e acesso. Vínculos de perfil por empresa e exceções individuais resolvem permissões. A empresa atual orienta o escopo das consultas centrais.

Project é a entidade operacional central: cliente, responsáveis, datas, valores e encerramento. ProjectMember relaciona pessoas ao projeto com função e estado de acesso. AnalystProjectRate representa tarifas e períodos; Task representa a atividade visível na interface, com participantes em task_participants; WorkLog representa apontamento de horas. ProjectExpense, ProjectTax e TaxType compõem o financeiro. ProjectInvitation e UserActivationToken cuidam de acesso temporário/ativação. AuditLog registra ações. ProjectDocument, CalendarEvent e InboxItem apoiam a rotina. Produtos, categorias e movimentos compõem estoque.

Não somar uma tarefa duas vezes: o cálculo identifica tarefas já representadas em work_logs por task_id, excluindo sua contribuição direta adicional. Não dividir o custo pelo número de coparticipantes: na implementação, o responsável é a base de precificação da tarefa; participantes representam colaboração, não rateio de trabalho.

Tarifas sobrepostas são rejeitadas para o mesmo analista/projeto. WorkLog exige vínculo ativo e tarifa válida na data. Tarefa pode ser criada sem tarifa e aparecer como pendência; cadastrar tarifa pode precificar tarefas pendentes, com auditoria. Esse comportamento precisa ser explicado como escolha para permitir registro operacional sem apresentar resultado financeiro completo.

10. Guia técnico de cada tela e funcionalidade

Em cada item: objetivo, caminho de código, comunicação, regra e decisão de desenvolvimento. Os caminhos são relativos à raiz /home/alexander/Documentos/Spot.

10.1 Login, recuperação e primeiro acesso

Interface: resources/js/App.jsx, componentes LegacySpotApp, FirstAccessPage, ForgotPasswordPage, ResetPasswordPage e AcceptInvitationPage. O nome LegacySpotApp não torna a API principal JWT: o login da SPA usa a rota de sessão.

Backend: Api/AuthController.php, Api/FirstAccessController.php, controllers de Auth/, Domain/Identity/Actions/CreateActivationLink.php.

Fluxo: administrador provisiona, conta fica pendente, link ativa e define senha; conta ativa entra por sessão. OAuth valida identidades autorizadas. Não existe cadastro público. A UI pede código adicional quando o login por senha informa exigência de 2FA. A decisão é controlar quem entra antes de autorizar recursos internos.

10.2 Shell, navegação, pesquisa e aplicativos

App.jsx: Dashboard, Sidebar, navGroups, pageMeta, popovers e busca. app.jsx monta a aplicação; api.js centraliza chamadas. O estado activePage seleciona a área e os popovers mantêm interações do topo. A pesquisa consulta dados acessíveis e não é um motor de indexação de todos os arquivos. Os atalhos externos usam nova aba e proteções de opener.

Decisão: manter navegação e identidade visual consistentes. Compromisso: App.jsx está grande; extrair telas e navegação para módulos menores facilitaria manutenção. Não afirmar uso de TypeScript: o frontend está em JavaScript/JSX.

10.3 Home

App.jsx e Api/HomeDashboardController.php; GET /api/v1/home-dashboard, PATCH de conclusão e prioridade. Consulta os projetos da empresa e, conforme autorização, os atribuídos. Mostra tarefas próprias/participadas em certos perfis. Resumo limitado a 20 projetos e 30 tarefas, não a carteira inteira. Mutações de tarefa revalidam projeto e autorização e geram auditoria.

Decisão: resumo para ação rápida. Demonstração: abrir uma tarefa, explicar responsável versus participantes e mostrar seu histórico; não alterar um projeto real para exemplificar.

10.4 Projetos e novo projeto

App.jsx/NewProjectPage, Management.jsx/ProjectWorkspace; ProjectController, SaveProjectRequest, ProjectFields, Project e ações FinalizeProject, ReopenProject, DeleteProject. Endpoints /projects, /projects/{id}/configuration, /finalize e /reopen.

Valida proposta, status, situação, contrato e referências da empresa. Campos financeiros são omitidos na resposta quando falta autorização, além dos controles da interface. Algumas permissões de campo têm autorização por exceção com padrão permissivo; não são uma política universal de negar todo campo por padrão.

Finalização exige motivo, registra autor/data e revoga vínculos ativos. Reabertura exige motivo e não reativa esses vínculos. Projetos finalizados têm destaque vermelho. Os bloqueios são fortes nos fluxos centrais, mas não afirmar bloqueio universal de todo módulo: o fluxo de documentos, por exemplo, não verifica finalização.

10.5 Detalhes financeiros e configuração do projeto

Management.jsx: ProjectWorkspace, Editor, FinancialResult. Backend: ProjectConfigurationController, ManagementController, ProjectExpenseController,

FinancialReportController. Domínio: TaskCosting, ReviewProjectExpense, ProjectFinancialSummary.

Abas/formulários trabalham membros, tarifas, impostos, despesas e relatório. Tarifas exigem membro ativo e vigência sem conflito. Composição tributária é substituída com auditoria. Despesas editadas voltam à aprovação. Só aprovadas entram no resultado.

Decisão: manter o contexto financeiro junto do projeto. Relatórios exigem financial.view, e a API financeira agrega a carteira da empresa; não assumir que ela sempre usa o mesmo filtro de projetos atribuídos da lista operacional.

10.6 Tarefas e detalhes

TasksPage.jsx, TaskDetails.jsx, AuditTrail.jsx; TaskV1Controller, Task, TaskCosting; /tasks e /history?project_id=...&task_id=.... A migração 2026_10_26_150000_create_task_participants_table.php adiciona participantes.

A tela carrega tarefas e opções; busca e filtros são aplicados no cliente à coleção recebida. Formulário converte horas/minutos para duration_minutes; valida de 1 a 1.440 minutos por registro. Há descrição, tipo, data, hora extra, responsável e coparticipantes. O título pode ser derivado da descrição. A tela confirma exclusão e usa dialog para detalhes/edição.

API valida pessoas ativas e participantes do projeto, regras próprias/de terceiros, status e projeto finalizado. A versão atual permite atribuição de responsável no contexto autorizado do projeto; o README antigo que associa toda atribuição somente a work_logs.create_for_others não descreve integralmente o comportamento atual de ensureAnalyst. Não confundir essa regra de tarefa com a API separada de work logs.

Decisão: quadro, lista e calendário são visões dos mesmos registros; não três bancos diferentes. Editar apenas descrição não deveria alterar o preço histórico; mudanças de data/analista/projeto/hora extra podem recapturar tarifa, conforme o código.

10.7 Equipes

App.jsx/TeamsPage, Api/TeamController.php, ProjectMember, ProjectConfigurationController. Rotas /teams, /teams/{project} e membros do projeto. Exibe nome da equipe, pessoas e funções. Vínculos são registros próprios e podem ser desativados.

Decisão: uma equipe é contextual ao projeto; a mesma pessoa pode ter relações distintas em projetos diferentes. Não é um sistema completo de recursos humanos ou capacidade de alocação.

10.8 Clientes

App.jsx/ClientsPage, Management.jsx/ParametersWorkspace, ManagementController; /management/parameters/clients. Cadastro inclui nome, razão social, CNPJ, e-mail e telefone. CNPJ tem normalização de dígitos e validação de tamanho/unicidade; não afirmar verificação de autenticidade na Receita Federal ou cálculo completo dos dígitos só por esse trecho.

Arquivamento inativa e mantém referências históricas. Consultas filtram empresa e, em certos perfis, clientes dos projetos acessíveis. Decisão: separar cadastro de cliente dos campos textuais legados do projeto.

10.9 Portfólios e gráficos

App.jsx/ProjectAnalyticsPage, PortfolioDashboard.jsx, PortfolioChart.jsx, portfolioReports.js; ProjectAnalyticsController, FinancialReportController, scripts/project_analytics.py.

Laravel autoriza, consulta projetos da empresa, calcula resumo financeiro e envia JSON ao processo Python. Pandas normaliza, agrupa e devolve JSON. React renderiza cartões, tabelas e gráficos; portfolioReports.js prepara agrupamentos e indicadores. Preferências são locais por painel. useAutoRefresh.js atualiza a carteira e o detalhe financeiro a cada 30 segundos quando a tela está visível, ao recuperar foco e após alterações locais/entre abas, sem aplicar um filtro ainda não confirmado.

Há barras, colunas, linhas, área, pizza, rosca, radar, árvore, funil, cartões e tabela, além de indicador circular quando apropriado. Radar precisa de pelo menos três indicadores. Composições negativas recebem explicação e alternativa tabular. CSV permite reutilizar os valores.

Período seleciona projetos pela data de início ou criação e mostra os totais desses projetos. Não é filtro de fluxo de caixa dentro do mês. O backend financeiro usa decimal; os gráficos usam números comuns. Decisão: manter regras financeiras no servidor e apresentação configurável no cliente.

10.10 Agenda

App.jsx/CalendarPage, CalendarController, CalendarEvent; /calendar e /calendar/events. Consulta eventos próprios e externos no intervalo e cria em Spot/Google/Microsoft/Teams. APIs externas recebem tokens e dados do evento; o servidor renova credenciais quando necessário e percorre as páginas de consulta, com limites e aviso de mais resultados. Next links Microsoft só podem apontar para HTTPS no host Graph. A criação normaliza o fuso do usuário para UTC e a conexão PostgreSQL fixa UTC. A tela consulta novamente a cada 60 segundos enquanto visível.

Decisão: reunir compromissos no contexto de trabalho. Limites: requer OAuth/consentimento e pode falhar por provedor; não há CRUD completo de eventos ou serviço contínuo de sincronização demonstrado. A criação externa e gravação local não formam uma transação distribuída; uma falha entre etapas exige tratamento operacional.

10.11 Caixa de entrada

App.jsx/InboxPage e InboxPopover, InboxController, InboxItem. Rotas de consulta, leitura, leitura em lote, arquivo, limpeza e restauração. O usuário vê seus próprios itens, atualizados automaticamente a cada 30 segundos enquanto a tela está visível. Mensagens arquivadas são restauráveis na janela de 30 dias e depois elegíveis a expurgo.

Decisão: concentrar avisos e leitura. Não existe envio de conversa/chat completo nesse controlador. O scheduler possui inbox:prune às 03h15 no fuso do aplicativo.

10.12 Meus documentos

App.jsx/DocumentsPage, ProjectDocumentController, ProjectDocument; /documents, /{id}/download. Arquivos até 10 MB e lista de formatos permitidos. Armazenamento privado local e metadados no banco. Se a transação de criação falha, o controlador tenta remover o arquivo já salvo.

Download/exclusão verificam proprietário; listagem também é pessoal. A associação de projeto não libera automaticamente os arquivos à equipe. O histórico registra operação e metadados, não conteúdo. Limites atuais: sem versionamento, edição colaborativa, antivírus ou bloqueio explícito de upload por projeto finalizado nesse fluxo.

10.13 Histórico geral e por entidade

App.jsx/HistoryPage, AuditTrail.jsx, resources/css/audit.css; HistoryController, AuditRecorder, AuditLog. Paginação por cursor before, filtros de projeto/tarefa e até 100 eventos por página. Contexto preserva item/autor nos eventos novos. audit.view amplia consulta da empresa; sem ela o usuário vê suas ações. Detalhes old/new e request_id exigem security.manage.

Decisão: rastrear ações sem expor detalhes a todos. Logs antigos não são reconstruídos; retenção remove eventos vencidos. Não apresentar o histórico como inviolável ou como auditoria de cada clique.

10.14 Perfil e avatar

App.jsx/ProfilePage, ProfileController, ProfileAvatarController; PATCH /profile, POST/DELETE /profile/avatar. Atualiza dados e imagem do usuário. Fuso horário é usado em recursos da conta. O avatar tem validação própria e persistência de arquivo/URL conforme o fluxo.

Decisão: manter identificação pessoal separada do cadastro de acesso administrativo. A tela de perfil não deve conceder a si mesma funções administrativas.

10.15 Segurança da conta

App.jsx/SecurityPanel, AccountSecurityController, AuthController, User. Rotas /security/* para senha, sessões, provedores e 2FA. Alterar senha exige confirmação da atual e encerra outras sessões. TOTP usa Google2FA e QR Code; códigos de recuperação são exibidos e armazenados por hash, com estrutura criptografada no modelo.

Decisão: oferecer proteção e revogação de acesso no próprio produto. A evidência de 2FA cobre o login por senha; não afirmar cobertura idêntica de todos os caminhos OAuth e JWT. A presença desses recursos não é certificação de conformidade.

10.16 Preferências de notificações

App.jsx/NotificationPanel, NotificationPreferenceController. Eventos/canais variam conforme permissão; o usuário salva e restaura preferências, resumo e silêncio. O catálogo passou a apresentar apenas eventos com produção implementada; opções antigas de comentários e revisão de arquivos foram removidas dessa configuração.

Decisão: o domínio app/Domain/Notifications/ liga operações auditadas a uma outbox persistente, em notification_deliveries, sem SMTP na requisição de negócio. DueReminders gera prazos de tarefas, compromissos locais a até 30 minutos e convites a até 24 horas da expiração. NotificationCenter aplica canais, fuso, silêncio e resumos; SendNotificationBatch revalida acesso e entrega pela fila. Chaves únicas e locks evitam duplicações normais. Diário é um lote por dia e semanal inicia na segunda-feira; avisos posteriores ao lote seguem ao próximo período. Lotes têm até 500 eventos. Eventos externos nunca registrados no Spot não recebem lembrete local. A entrega externa requer SMTP e ativação da configuração; notificações internas funcionam independentemente dela.

10.17 Integrações e Apps

App.jsx/AppsPopover, IntegrationsModal, controllers Google/Microsoft em Auth/, configuração config/services.php, tokens criptografados no modelo User. Conectar encaminha para o provedor; desconectar remove credenciais do Spot. A API informa se a aplicação está configurada, se a conta está vinculada e se há credencial de calendário, sem devolver segredos. A tela diferencia falta de configuração e necessidade de reconectar.

Decisão: delegar autenticação à conta corporativa e aproveitar calendários. Os atalhos Drive/Gmail/Word/Excel e outros são links; não há edição de seus dados dentro do Spot apenas por estarem no menu.

10.18 Administração de usuários e permissões

App.jsx (área administrativa), Management.jsx/ParametersWorkspace, AdminUserController, ManagementController, User, Profile, Permission, ProjectFields, AppServiceProvider.

Administrador cria/ativa/desativa usuários e perfis; último administrador ativo é protegido. hasPermission considera administrador, override individual e permissões dos perfis da empresa. Gate::before integra habilidades com ponto ao mecanismo. Campos do projeto recebem direitos view/edit. Não confundir esconder botão com autorizar no servidor: ambas as camadas são necessárias.

Decisão: atender reclamações de acesso indiscriminado do sistema anterior. Há escopo multiempresa no núcleo, mas não prova de cobertura uniforme de todos os módulos.

10.19 Parâmetros, retenção e diretório Microsoft

Management.jsx/ParametersWorkspace, ManagementController, DirectoryController, CompanySetting; rotas /management/settings, /management/parameters/*, /management/directory e /import.

Tipos de atividade, despesa, imposto, situação e status são cadastros da empresa. Retenção de auditoria permite 1 a 120 meses, padrão 12. routes/console.php agenda audit:prune às 03h00 no fuso do aplicativo. Diretório Graph exige tenant/client/segredo e User.Read.All de aplicação; importação é explícita, como analista, sem elevar a administrador.

Decisão: reduzir mudanças de código para manter opções de negócio e políticas. Login Microsoft e consulta de diretório são configurações/consentimentos distintos.

10.20 Convites

App.jsx/ProjectInvitationModal e AcceptInvitationPage; ProjectInvitationController, CreateProjectInvitation, AcceptProjectInvitation, ProjectInvitation, SendAccessEmail, ProjectInvitationMail.

Convite concede escopo de projeto, validade de até sete dias e aceite único; token exibido na criação e armazenado como SHA-256. Conta existente precisa provar identidade por login; convite pode exigir nova senha. Revogação impede uso futuro.

E-mail usa fila emails, payload criptografado, até três tentativas, revalidação do link e agendamento após commit. Decisão: acesso externo controlado, sem transformar convite em acesso geral à empresa. “Convite criado” não significa “e-mail entregue”.

10.21 Apontamentos de horas por API

WorkLogController, CreateWorkLog, WorkLog, AnalystProjectRate; GET/POST /projects/{id}/work-logs. Exige projeto mutável, membro ativo e tarifa da data. Persiste custo e venda do período. Há API própria; a principal tela operacional usa tarefas. Não inventar uma tela autônoma completa para cada endpoint existente.

Decisão: distinguir registro de trabalho de organização de tarefa e preservar compatibilidade com o modelo financeiro. A ligação por task_id evita contagem dupla.

10.22 Estoque

InventoryApp.jsx e seleção /estoque em App.jsx; CategoryController, ProductController, MovementController, StoreStockMovementRequest, RegisterStockMovement, Product, StockMovement.

Saldo só muda via movimentação após criação inicial; saída maior que saldo é rejeitada. A ação usa transação e bloqueio da linha e admite repetição em conflitos transacionais. Existem endpoints de sessão e JWT legado.

Decisão: registrar saldo e histórico juntos. Limites: o CRUD examinado não aplica o mesmo filtro por empresa nem permissões granulares dos projetos. Não foi encontrada baixa de estoque que vire automaticamente despesa/custo de projeto. Apresentar como módulo complementar.

11. Exemplo completo: do clique ao resultado financeiro

Exemplo didático, não dados de cliente: projeto de R$ 10.000,00; duas horas normais com custo de R$ 100/h; uma hora extra com custo de R$ 150/h e venda de R$ 200/h; despesa aprovada de R$ 300,00; imposto de 10% sobre receita total e comissão de 5% sobre bruto.

Receita gerencial: 10.000 + 200 = R$ 10.200,00. Mão de obra: 2 × 100 + 1 × 150 = R$ 350,00. Imposto: R$ 1.020,00. Comissão: R$ 510,00. Custo operacional: 350 + 300 = R$ 650,00. Lucro: 10.200 − 1.020 − 510 − 650 = R$ 8.020,00. Margem: aproximadamente 78,63%.

O valor é gerencial com receita contratual como base, não caixa recebido. Se a comissão fosse líquida de impostos, a base seria R$ 9.180 e a comissão R$ 459: isso mudaria o resultado. O sistema permite distinguir essas bases.

Caminho de uma tarefa: TasksPage.save converte horas em minutos → sessionRequest obtém CSRF → POST /api/v1/tasks → middleware de sessão → TaskV1Controller.store valida e verifica projeto/pessoas → transação cria tarefa e participantes → TaskCosting.capture busca tarifa vigente e salva snapshot → AuditRecorder.record registra histórico → JSON atualiza a tela. Quando o financeiro é consultado, ProjectFinancialSummary.calculate soma os registros válidos, evita duplicidades e informa pendências.

Se um passo dentro da transação de banco falhar, tarefa/participantes/auditoria desse fluxo são revertidos. Arquivos, SMTP e APIs externas exigem mecanismos específicos: não participam magicamente do mesmo commit do PostgreSQL.

12. Tecnologias e versões declaradas

Backend: PHP ^8.3, Laravel ^13.8, Sanctum ^4.3, Socialite ^5.29, provider Microsoft, JWT Auth ^2.3, Google2FA e QR Code. Persistência: PostgreSQL, com imagem postgres:17-alpine no Compose; Eloquent e migrações. Frontend: React ^18.3.1, Vite ^8.0.0, JavaScript/JSX, CSS próprio, Lucide e Iconify. Node declarado ^22.12.0; a linha do Node é 22. Análise: Python e Pandas >=2.2,<3. Cálculo financeiro: Brick Math/BigDecimal. Testes: PHPUnit ^12.5.12 e Playwright. Execução: FrankenPHP/Caddy, Composer/npm e Docker/Compose.

Intervalos dos manifestos não são garantia de patch instalado nem “última versão”. O build desta tarefa identificou Vite 8.3.5. Não há dependência de React Router, framework de CSS ou biblioteca de IA identificada como base do produto.

13. Testes, evidências e limites da validação

Em 26/10/2026 foram executados ./php vendor/bin/phpunit: 122 testes aprovados, 788 asserções, duração reportada 17.437 ms. O npm run build também concluiu, com 1.583

módulos transformados. A configuração PHP usa PostgreSQL e schema spot_testing, além de transporte de e-mail de teste. Não foram usados cadastros reais como massa de teste.

A suíte existente cobre autenticação, recuperação, perfil, conta, permissões, usuários, projetos, equipes, tarefas, convites, horas, despesas, financeiro, agenda, estoque, auditoria e outros fluxos. Scripts Playwright existentes cobrem interface, idiomas, tutorial, administração e portfólio, frequentemente com API simulada. Capturas deste material usam esse mesmo princípio de isolamento.

Os testes novos cobrem entregas, resumos, silêncio, deduplicação, revogação, falha SMTP, paginação de calendários, renovação de tokens e fusos. test-live-updates.mjs verifica atualização por timer/eventos/abas, filtros não aplicados e limpeza ao sair. Asserção é uma verificação feita pelo teste, e não sinônimo de funcionalidade. Passar 122 testes não significa ausência de defeitos, 100% de cobertura, usabilidade comprovada ou homologação externa. Não foi executado ensaio de carga, auditoria ofensiva, avaliação formal de acessibilidade ou teste real de entrega SMTP/OAuth nesta tarefa. Os resultados de navegador efetivamente executados nesta entrega estão em assets/validacao.txt.

14. Implantação e manutenção

iniciar.sh inicia PHP/FrankenPHP na porta 8000, Vite na 5173, worker e scheduler. A aplicação usa PostgreSQL; credenciais e APP_KEY ficam na configuração de ambiente e não devem ser incluídas em slides. O Python pode ser selecionado por PYTHON_BINARY. O comando notifications:dispatch é agendado a cada minuto; notifications:status apenas consulta pendências. A nova migração de notificações foi aplicada ao ambiente local sem seed. SPOT_ACTIVITY_EMAIL_ENABLED inicia em false para não disparar mensagens externas na implantação: exige SMTP homologado e ativação explícita da configuração para e-mails de atividades. Eventos enquanto desativado não formam campanha de e-mail retroativo. Convites/ativação preservam sua fila anterior. O worker revalida permissão e estado antes do envio.

A imagem Docker faz build do frontend em Node e instala dependências PHP/Python para execução. O Compose usa web, worker, scheduler e database, com volumes para dados e arquivos. Caddy fornece a camada HTTP/HTTPS conforme domínio e configuração. O processo roda como usuário não root na imagem.

Para produção: domínio/HTTPS, APP_DEBUG=false, cookies adequados, migrações controladas, SMTP, worker supervisionado, scheduler único, backup e restauração testada, monitoramento de falhas, rotação de segredos e atualizações. A presença de Docker não comprova que um servidor de produção já esteja implantado ou escalável.

Comandos de estudo: ./php artisan route:list, ./php vendor/bin/phpunit, npm run build, ./php artisan schedule:list, ./php artisan queue:status e ./php artisan audit:prune --dry-run. O expurgo sem dry-run apaga dados elegíveis; não deve ser parte de uma demonstração improvisada.

15. Ajustes necessários entre monografia e produto atual

1. Substituir SQLite por PostgreSQL nos trechos que descrevem a versão atual. Preservar SQLite só se houver um relato histórico explicitamente datado.

2. Revisar resumo, abstract, objetivos e conclusões que apresentam IA como entregue. Não há serviço de inferência do produto identificado. Uso de ferramentas de desenvolvimento não é integração de IA ao Spot.

3. Atualizar números históricos: 52 testes/262 asserções e 105 rotas pertencem à documentação anterior. Nesta execução há 122 testes/788 asserções; o inventário atual de rotas está no anexo, com critério separado de rotas do framework.

4. Descrever portfólio como código local implementado e validado nesta data. Estar ou não commitado/publicado é outra questão; não inferir implantação a partir de código presente.

5. Não apresentar AWS como concorrente direto de tarefas: é infraestrutura possível para hospedagem.

6. Não apresentar preços históricos como atuais ou comparar licenças sem custo total e unidade de cobrança.

7. Corrigir referências de pesquisa: conferir totais por figura, 57 comentários versus 49 frequências e ausência de tabela cruzada.

8. Diferenciar integração efetiva de atalhos, painéis de consulta de tempo real por push, catálogo de notificações de funcionalidades externas ao escopo, documentos pessoais de biblioteca colaborativa.

9. Distinguir lucro gerencial de caixa/contabilidade e adequação ao cliente de superioridade comprovada sobre concorrentes.

10. Harmonizar a identificação da licença entre os manifestos e o texto, sem prometer cessão contratual que não foi analisada.

16. Roteiro oral — 15 slides, 14min10s

Cada bloco é uma base de fala. Os tempos incluem apontar para o gráfico ou imagem; o texto deve ser ensaiado em voz alta. Não abrir uma demonstração ao vivo adicional dentro dos mesmos 15 minutos. Se quiser demonstrar o sistema, substituir a exposição das telas, mantendo o tempo total.

Slide 1 — SPOT: pesquisa que virou sistema — 20 s

“Somos a equipe do Spot, Sistemas de Projetos, Organização e Tarefas. O projeto nasceu de uma necessidade da Computécnica: organizar a execução dos projetos com mais clareza e ligar esse trabalho ao seu resultado financeiro. Vamos mostrar como a pesquisa orientou as decisões e como elas aparecem no sistema.”

Slide 2 — Problema real e duas pesquisas — 50 s

“O ponto de partida foi um sistema com visual antigo, dificuldades de integração e controle insuficiente de responsabilidades e custos. Usamos duas bases: a pesquisa de campo, para identificar dores e expectativas, e o estudo de sistemas, para conhecer soluções existentes. O formulário reuniu 94 respostas entre março e agosto de 2026. A amostra é de conveniência: orienta nosso projeto, mas não representa estatisticamente todas as empresas. Também confrontamos essas evidências com o memorial e as necessidades específicas do cliente.”

Slide 3 — Quem respondeu e onde já trabalha — 60 s

Leitura do gráfico de linhas: cada ponto representa uma faixa etária, com percentual calculado sobre 94. A ligação mostra o perfil das faixas ordenadas; não representa evolução temporal nem acompanhamento das mesmas pessoas.

“Dos 94 participantes, 88 estavam empregados. A amostra reúne diferentes idades e 65 pessoas que não atuam diretamente em gestão de projetos. Isso nos ajudou a pensar no uso cotidiano, além de especialistas. O Office 365 teve 52 menções e 17 pessoas disseram não utilizar software. As ferramentas podem aparecer juntas, por isso essas contagens não somam 94. No Spot, essa presença do Microsoft se traduz em login, agenda e integração de diretório; a diversidade de usuários levou à prioridade de interface simples e tutorial.”

Slide 4 — As dores que orientaram o núcleo — 70 s

“Os dois números centrais são 63,8% com dificuldade de organização e 88,3% que percebem relação entre organização e faturamento. Também encontramos 41,5% com dificuldade de alinhamento e 31,9% com dificuldade de registrar custos adicionais. O Spot responde com tarefas e responsáveis, equipes, despesas revisadas e resultado financeiro. Esses números são percepções da amostra, não provas de que o Spot aumentou faturamento. Usamos 94 como base e sinalizamos que algumas figuras não discriminam todos os participantes.”

Slide 5 — Existe espaço para uma solução dedicada — 65 s

Leitura do gráfico de setores: concordância = 43/94 (45,7%); talvez = 33/94 (35,1%); não = 14/94 (14,9%); diferença documental = 4/94 (4,3%). A última fatia é apenas a diferença entre a base declarada e as categorias publicadas, não uma resposta observada. As legendas também mostram as contagens.

“Sobre software de projetos, 33 não têm e 27 usam uma solução cujo foco não é esse. Ao perguntar sobre uma ferramenta dedicada, 43 concordam diretamente e 33 respondem talvez. Assim, 80,9% demonstram concordância ou abertura, mas não intenção comprovada de compra. O estudo também identifica fluxos internos e externos e opiniões intermediárias sobre a interface atual. Isso levou a centralizar projetos e criar diferentes visões de trabalho. O suporte a contextos internos e externos tem base no modelo, mas não afirmamos dois fluxos completos e separados.”

Slide 6 — O que os participantes pediram — 70 s

“A análise aberta destacou interface simples, financeiro, integrações, indicadores e tarefas com agenda. Mostramos a ligação de cada tema com a entrega: tutorial e navegação; horas, tarifas e despesas; calendários; portfólio; quadro, lista e calendário de tarefas. Os alertas agora são gerados por prazo e evento, com caixa de entrada, fila de e-mail, resumos e horário de silêncio. O portfólio atualiza automaticamente. Serviços externos continuam exigindo credenciais, consentimento e SMTP; isso é condição de operação, não ausência do código. A tabela da pesquisa soma 49 frequências e o texto cita 57 comentários: registramos essa diferença e não a preenchemos com respostas inventadas.”

Slide 7 — Referências do mercado e nosso recorte — 75 s

“Estudamos Monday, Asana, Jira, Trello, ClickUp e Notion, além das ferramentas brasileiras e de automação citadas no trabalho. Eles já oferecem recursos fortes de organização, colaboração e automação. O diferencial que defendemos não é exclusividade de quadros ou dashboards: é entregar um fluxo específico da Computécnica, com tarifas por vigência, despesas aprovadas e regras de encerramento em código próprio. TaskRush também trabalha com margem, então rentabilidade não é invenção nossa. AWS é infraestrutura, não concorrente direto. Código próprio dá autonomia de evolução, mas continua exigindo hospedagem e manutenção.”

Slide 8 — Do contrato à margem — 65 s

“No núcleo financeiro, receita gerencial é contrato mais horas extras faturáveis. Dela saem impostos, comissão, mão de obra e despesas aprovadas. A tarifa é capturada na atividade, preservando o histórico quando há reajuste. Se faltar tarifa, o resultado é marcado como parcial. Neste exemplo didático, R$ 10.200 de receita, menos tributos, comissão, mão de obra e despesas, resultam em R$ 8.020. O cálculo usa aritmética decimal. Não chamamos isso de fluxo de caixa: não há comprovação de recebimento bancário.”

Slide 9 — Organização na tela de trabalho — 60 s

“Estas são telas reais com dados demonstrativos. A Home resume o que merece atenção. Em tarefas, quadro, lista e calendário mostram os mesmos registros. Cada atividade tem responsável, colaboradores, data, tipo e duração. O histórico permite consultar mudanças. Não basta esconder um botão: a API também valida acesso e estado do projeto. Tutorial, temas e três idiomas procuram reduzir a barreira de entrada; a facilidade de uso ainda precisa ser medida em testes com usuários.”

Slide 10 — Indicadores para decidir — 55 s

“O portfólio reúne a carteira e permite escolher indicadores, agrupamentos e gráficos. É possível comparar valores planejados e realizados e exportar CSV. O Laravel aplica a autorização e envia dados em JSON ao Python com Pandas; o script devolve os indicadores para o React. Preferências são salvas no navegador. A tela atualiza a cada 30 segundos quando visível e depois de alterações locais ou entre abas, mantendo os filtros aplicados. O período seleciona projetos pela data de início ou criação: não é um extrato de entradas financeiras do mês. A atualização é por consultas, não WebSocket.”

Slide 11 — Governança e integração no fluxo — 60 s

“O controle de acesso combina perfis, permissões e vínculo com projeto. Finalizar exige justificativa, bloqueia os principais lançamentos e revoga vínculos ativos; reabrir não restaura esses vínculos automaticamente. A auditoria registra ações relevantes e possui retenção configurável. Na integração, o que existe de fato é OAuth, calendários, criação de Teams e diretório Microsoft, com configuração externa. Os 24 ícones de aplicativos são atalhos. Convites usam links com validade, uso único e envio por fila.”

Slide 12 — Como a arquitetura sustenta isso — 60 s

“O Spot é um monólito organizado por camadas. React apresenta a interface; a API Laravel valida acesso e dados; ações de domínio executam regras; Eloquent persiste no PostgreSQL. Python faz a agregação analítica, sem acesso direto ao banco. Worker processa e-mails e scheduler executa manutenção. A escolha mantém uma implantação administrável para a equipe, sem exigir microserviços. Separar responsabilidades ajuda a testar e evoluir, embora alguns controllers e o componente principal ainda possam ser divididos melhor.”

Slide 13 — Uma tarefa, do clique ao banco — 55 s

“Ao salvar, TasksPage transforma horas e minutos em um número inteiro. O cliente de API prepara CSRF e envia a requisição. TaskV1Controller verifica o projeto e as pessoas. Dentro da transação, cria a tarefa, vincula participantes, captura tarifa e registra auditoria. O JSON atualiza a tela. Quando o relatório é aberto, o cálculo usa essa fotografia e evita somar novamente tarefas que já tenham apontamento associado. Esse caminho mostra que a funcionalidade existe além do desenho da tela.”

Slide 14 — O que validamos e o que falta — 50 s

“Nesta versão local, os 122 testes passaram, com 788 asserções, e o build de produção concluiu. Isso dá evidência dos comportamentos cobertos, não garantia de ausência de problemas. Completamos as notificações de atividade, os lembretes de tarefas/agenda e a atualização automática dos painéis. Permanecem a homologação com usuários e provedores reais e os ensaios de carga; o envio externo exige a configuração de operação. A monografia precisa refletir PostgreSQL e a versão técnica mais recente. A honestidade sobre o escopo evita defender algo que o código não demonstra.”

Slide 15 — Contribuição do trabalho — 35 s

“O trabalho conecta três coisas: uma necessidade real, pesquisa para priorizar requisitos e implementação verificável. O Spot organiza a execução e a relaciona ao resultado financeiro, com governança e autonomia de evolução. Nossa contribuição é uma solução adaptada à Computécnica e uma base técnica que pode amadurecer com uso e validação. Obrigado.”

17. Perguntas prováveis da banca

Por que desenvolver se já existe Trello, Asana ou ClickUp? Porque o objetivo foi atender regras e contexto do cliente e aplicar conhecimentos em uma solução controlável. Sistemas existentes são referências e alternativas reais; a decisão precisa considerar adequação e custo total, não alegar que não fazem gestão.

Qual é o diferencial mais concreto? A combinação, no fluxo do cliente, de tarefas, tarifas históricas, despesas aprovadas, planejado × realizado e transições auditadas. É adequação de domínio; cada recurso isolado pode existir em outros produtos.

A pesquisa prova demanda de mercado? Identifica padrões e necessidades na amostra de conveniência. Não comprova mercado total, compra, representatividade ou retorno sobre investimento.

Por que 94 pessoas e gráficos com menos respostas? O documento publicado não discrimina toda a diferença. Mantivemos os valores originais, denominador explícito e a ressalva; a solução correta é reconciliar com a exportação original, não inventar respostas.

Por que apresentar 80,9% e não 81% de compradores? Porque 43 concordam e 33 estão em dúvida. O agregado representa abertura; não compradores. O valor vem de 76/94, arredondado a uma casa decimal.

Como comprovar que é fácil? O código implementa mecanismos favoráveis à aprendizagem, mas a comprovação exige teste de usabilidade. Definir tarefas, perfis, taxa de sucesso, tempo e satisfação seria um próximo estudo.

Por que Python se o Laravel já calcula? Laravel mantém a regra financeira e a autorização; Pandas consolida análises. É uma separação conveniente para o módulo, mas há custo operacional. Não é a única arquitetura possível.

Por que PostgreSQL? É o banco adotado no estado atual, com transações e bloqueios úteis às operações. Sua escolha não significa que todo projeto pequeno precise dele; aqui também simplifica manter um único banco suportado.

Por que não microserviços? O domínio e a equipe não exigem esse custo de comunicação/implantação. Um monólito em camadas permite desenvolver e testar com menos infraestrutura; processos de fila não o tornam automaticamente microserviços.

Como manter resultado correto após reajuste? Tarifa com vigência é copiada para o registro. Reajuste posterior não altera a fotografia, exceto recaptura motivada por alteração relevante ou precificação pendente prevista na regra.

Concluir uma tarefa e finalizar um projeto são a mesma coisa? Não. Status de tarefa organiza execução; finalização do projeto é um estado governado, com justificativa, autor, bloqueios e efeitos de acesso.

A reabertura devolve acesso à equipe? Não automaticamente. A implementação preserva os vínculos revogados; o gestor precisa revisar as atribuições.

O lucro representa dinheiro no banco? Não. É resultado gerencial com contrato e horas extras como receita-base. Recebimento, conciliação e emissão fiscal não estão implementados como ERP completo.

O administrador acessa tudo de qualquer empresa? O desenho central dá poderes no contexto da empresa atual. Não se deve afirmar isolamento uniforme de todos os módulos: estoque é uma lacuna identificada.

Por que existe estoque no TCC de projetos? É módulo adicional/compatibilidade do repositório. Serve como exemplo de transação e concorrência, mas não é a justificativa principal da pesquisa de projetos nem está ligado automaticamente ao financeiro do projeto.

Como funcionam os avisos? Eventos de negócio geram entregas persistentes e um agendador avalia prazos a cada minuto. Canais, fuso e silêncio são revalidados; os e-mails usam fila e podem ser imediatos, diários ou semanais. É necessário ativar SMTP para envio externo. Não confundir alerta de agenda local com todos os eventos de calendários externos.

Os documentos são da equipe? O fluxo atual é pessoal por proprietário; associar projeto não libera o arquivo a todos os membros.

O sistema está em produção e homologado? Este material comprova código local, build e testes executados. Não confirma implantação produtiva, consentimentos de tenant, SMTP real ou desempenho sob carga.

122 testes significam 122 funcionalidades? Não. Testes verificam cenários; asserções verificam condições. A cobertura e a qualidade dependem dos casos escolhidos.

Como a segurança é garantida? Há medidas concretas — sessão/CSRF, autorização, hash de senha, proteção de tokens, 2FA, limites e auditoria —, mas não garantia absoluta. Validar todos os caminhos e o ambiente de produção continua necessário.

O sistema é gratuito? O repositório não impõe assinatura SaaS por usuário para rodar esse código, mas infraestrutura, suporte e manutenção custam. Não foi calculada economia comprovada nem verificada cessão contratual.

18. Próximos passos com critério de conclusão

1. Reconciliar a exportação original do Forms e a codificação aberta; publicar tabelas com n por pergunta e regras claras para múltiplas respostas.

2. Validar usabilidade com tarefas reais e usuários da Computécnica; medir sucesso, tempo e satisfação antes/depois.

3. Homologar Google/Microsoft, expiração/renovação, Graph, criação de Teams e envio SMTP em ambiente controlado.

4. Operar e acompanhar as notificações implementadas: revisar destinatários, homologar SMTP, habilitar o envio de atividades, supervisionar worker e scheduler e acompanhar falhas. Os canais, eventos, resumos e silêncio têm testes automatizados.

5. Revisar isolamento e autorização do estoque; definir se terá vínculo financeiro com projeto.

6. Medir portfólio com grande volume, reduzir consultas repetidas e avaliar cache/execução analítica se necessário.

7. Melhorar modularização do frontend e consistência de permissões entre tela, endpoints e documentação.

8. Preparar produção com backup restaurável, monitoramento, supervisão de filas e plano de atualização.

19. Material de apoio e demonstração

Arquivos entregues: apresentação PPTX editável, PDF dos slides, este guia em Markdown/TXT/DOCX/PDF, roteiro de tempo, dados JSON/CSV, gráficos PNG/SVG e capturas da interface. Slides usam gráficos de pesquisa como vetores/elementos editáveis quando construídos no PowerPoint; gráficos completos de apoio estão exportados como imagens e SVG.

Demonstração curta opcional em substituição aos slides de tela: abrir Home; entrar em Minhas tarefas; mostrar responsável/participantes e histórico; abrir Portfólios; explicar planejado × realizado e pendência de tarifa. Preparar dados fictícios e não depender de envio de e-mail ou autorização OAuth ao vivo. Se a conexão falhar, usar as capturas/PDF já entregues.

Divisão sugerida para quatro integrantes: slides 1–3 (problema/método/contexto); 4–7 (resultados e comparação); 8–11 (funcionalidades/diferenciais); 12–15 (arquitetura,

validação e conclusão). Cada pessoa deve conhecer também o caminho de uma tarefa e as limitações metodológicas.

20. Referências e procedência

EQUIPE SPOT. Capítulo de levantamento de pesquisa de campo. 2026. Documento DOCX fornecido pelo usuário, arquivo /home/alexander/Downloads/Capitulo_Levantamento_Pesquisa_ABNT.docx. Atribuição operacional ao projeto; autoria específica do capítulo não foi individualizada no arquivo analisado.

LONGO, Alexander Molany; SOARES, Lucas Rodrigues; MATOS, Pedro Galvão de; SOUZA, Yasmin Regina de. SPOT: Sistemas de Projetos, Organização e Tarefas. Santo André: Colégio Dr. Clóvis Bevilacqua, 2026. Monografia em elaboração, arquivo Monografia_SPOT_documentacao_integrada.docx.

SPOT. Código-fonte e documentação local. 2026. Diretório /home/alexander/Documentos/Spot; inclui alterações locais não commitadas. Consulta e validação: 26 out. 2026. Fontes técnicas principais: README.md, docs/MAPA-DO-CODIGO.md, docs/memorial/levantamento.md, manifestos, rotas, controllers, domínio, modelos, migrações e testes.

MONDAY.COM. Gestão de projetos e plataforma. Disponível em: https://monday.com/w/use-cases/ai-project-management-software. Acesso em: 26 out. 2026.

ASANA. Features. Disponível em: https://asana.com/features. Acesso em: 26 out. 2026.

ATLASSIAN. Jira features. Disponível em: https://www.atlassian.com/software/jira/features. Acesso em: 26 out. 2026.

TRELLO. Tour. Disponível em: https://trello.com/tour. Acesso em: 26 out. 2026.

CLICKUP. Features. Disponível em: https://clickup.com/features. Acesso em: 26 out. 2026.

NOTION. Projects. Disponível em: https://www.notion.com/product/projects. Acesso em: 26 out. 2026.

TASKRUSH. Gerenciamento de projetos. Disponível em: https://taskrush.com.br/gerenciamento-de-projetos/. Acesso em: 26 out. 2026.

ARTIA. Inteligência artificial para projetos e cronogramas. Disponível em: https://artia.com/produto/ia-do-artia/. Acesso em: 26 out. 2026.

JESTOR. IA e MCP. Disponível em: https://jestor.com/pt/ai/. Acesso em: 26 out. 2026.

AMAZON WEB SERVICES. What is cloud computing? Disponível em: https://aws.amazon.com/what-is-cloud-computing/. Acesso em: 26 out. 2026.

As referências de mercado foram usadas para posicionamento funcional, sem reprodução de preços e sem ranking de desempenho. A apresentação usa referências abreviadas e o guia mantém os endereços completos. A organização em seções, fontes e referências

segue uma apresentação acadêmica; a formatação final do TCC deve seguir o modelo da instituição. Não foi alterada nem certificada a conformidade integral da monografia com normas ABNT.

Anexo A. Inventário atual de rotas

Inventário obtido por ./php artisan route:list --json: 139 entradas totais (incluindo framework). Subconjunto por critério automático de action fora de Laravel/Illuminate e sem healthcheck: 134. Esses números não são contagens de funcionalidades.

Método	URI	Ação
POST	/api/auth/login	Api\AuthController@login
POST	/api/auth/logout	Api\AuthController@logout
GET / HEAD	/api/auth/me	Api\AuthController@me
GET / HEAD	/api/categories	Api\CategoryController@index
POST	/api/categories	Api\CategoryController@store
PUT / PATCH	/api/categories/{category}	Api\CategoryController@update
DELETE	/api/categories/{category}	Api\CategoryController@destroy
GET / HEAD	/api/dashboard	Api\MovementController@dashboard
GET / HEAD	/api/movements	Api\MovementController@index
POST	/api/movements	Api\MovementController@store
GET / HEAD	/api/products	Api\ProductController@index
POST	/api/products	Api\ProductController@store
GET / HEAD	/api/products/{product}	Api\ProductController@show
PUT / PATCH	/api/products/{product}	Api\ProductController@update
DELETE	/api/products/{product}	Api\ProductController@destroy
GET / HEAD	/api/projects	Api\ProjectController@index
POST	/api/projects	Api\ProjectController@store
GET / HEAD	/api/projects/{project}	Api\ProjectController@show
PUT / PATCH	/api/projects/{project}	Api\ProjectController@update
DELETE	/api/projects/{project}	Api\ProjectController@destroy
GET / HEAD	/api/tasks	Api\TaskController@index
POST	/api/tasks	Api\TaskController@store
PUT / PATCH	/api/tasks/{task}	Api\TaskController@update
DELETE	/api/tasks/{task}	Api\TaskController@destroy
GET / HEAD	/api/v1/admin/users	Api\AdminUserController@index
POST	/api/v1/admin/users	Api\AdminUserController@store
PATCH	/api/v1/admin/users/{user}	Api\AdminUserController@update
DELETE	/api/v1/admin/users/{user}	Api\AdminUserController@destroy

POST	/api/v1/auth/login	Api\AuthController@loginSession
POST	/api/v1/auth/logout	Api\AuthController@logoutSession
GET / HEAD	/api/v1/auth/me	Api\AuthController@meSession
GET / HEAD	/api/v1/auth/me/projects	Api\AuthController@myProjects
GET / HEAD	/api/v1/auth/timezones	Api\AuthController@timezones
GET / HEAD	/api/v1/calendar	Api\CalendarController@index
POST	/api/v1/calendar/events	Api\CalendarController@store
GET / HEAD	/api/v1/categories	Api\CategoryController@index
POST	/api/v1/categories	Api\CategoryController@store
PUT / PATCH	/api/v1/categories/{category}	Api\CategoryController@update
DELETE	/api/v1/categories/{category}	Api\CategoryController@destroy
GET / HEAD	/api/v1/dashboard	Api\MovementController@dashboard
GET / HEAD	/api/v1/documents	Api\ProjectDocumentController@index
POST	/api/v1/documents	Api\ProjectDocumentController@store
DELETE	/api/v1/documents/{document}	Api\ ProjectDocumentController@destroy
GET / HEAD	/api/v1/documents/{document}/ download	Api\ ProjectDocumentController@download
PUT	/api/v1/expenses/{expense}	Api\ ManagementController@updateExpense
DELETE	/api/v1/expenses/{expense}	Api\ ManagementController@deleteExpense
POST	/api/v1/expenses/{expense}/review	Api\ProjectExpenseController@review
POST	/api/v1/first-access	Api\FirstAccessController@requestLink
POST	/api/v1/first-access/{token}	Api\FirstAccessController@activate
GET / HEAD	/api/v1/history	Api\HistoryController@index
GET / HEAD	/api/v1/home-dashboard	Api\HomeDashboardController@show
PATCH	/api/v1/home-dashboard/tasks/{task}/ priority	Api\ HomeDashboardController@updatePriority
PATCH	/api/v1/home-dashboard/tasks/{task}/ toggle	Api\ HomeDashboardController@toggleTask
GET / HEAD	/api/v1/inbox	Api\InboxController@index
DELETE	/api/v1/inbox	Api\InboxController@clear
PATCH	/api/v1/inbox/read-all	Api\InboxController@readAll
DELETE	/api/v1/inbox/{inboxItem}	Api\InboxController@archive

PATCH	/api/v1/inbox/{inboxItem}/read	Api\InboxController@read
PATCH	/api/v1/inbox/{inboxItem}/restore	Api\InboxController@restore
POST	/api/v1/invitations/{invitation}/revoke	Api\ProjectInvitationController@revoke
GET / HEAD	/api/v1/invitations/{token}	Api\ProjectInvitationController@showAcceptance
POST	/api/v1/invitations/{token}/accept	Api\ProjectInvitationController@accept
GET / HEAD	/api/v1/management/access	Api\ManagementController@access
GET / HEAD	/api/v1/management/directory	Api\DirectoryController@index
POST	/api/v1/management/directory/import	Api\DirectoryController@import
GET / HEAD	/api/v1/management/parameters/{type}	Api\ManagementController@parameters
POST	/api/v1/management/parameters/{type}	Api\ManagementController@saveParameter
PUT	/api/v1/management/parameters/{type}/{id}	Api\ManagementController@saveParameter
DELETE	/api/v1/management/parameters/{type}/{id}	Api\ManagementController@deleteParameter
GET / HEAD	/api/v1/management/settings	Api\ManagementController@settings
PUT	/api/v1/management/settings	Api\ManagementController@saveSettings
GET / HEAD	/api/v1/management/users/{user}/permissions	Api\ManagementController@permissions
PUT	/api/v1/management/users/{user}/permissions	Api\ManagementController@savePermissions
GET / HEAD	/api/v1/movements	Api\MovementController@index
POST	/api/v1/movements	Api\MovementController@store
GET / HEAD	/api/v1/notification-preferences	Api\NotificationPreferenceController@show
PUT	/api/v1/notification-preferences	Api\NotificationPreferenceController@update
DELETE	/api/v1/notification-preferences	Api\NotificationPreferenceController@reset
GET / HEAD	/api/v1/parameters	Api\ParameterController@index
GET / HEAD	/api/v1/products	Api\ProductController@index
POST	/api/v1/products	Api\ProductController@store
GET / HEAD	/api/v1/products/{product}	Api\ProductController@show
PUT / PATCH	/api/v1/products/{product}	Api\ProductController@update

DELETE	/api/v1/products/{product}	Api\ProductController@destroy
GET / HEAD	/api/v1/projects	Api\ProjectController@index
POST	/api/v1/projects	Api\ProjectController@store
DELETE	/api/v1/projects/{project}	Api\ ProjectWorkflowController@destroy
GET / HEAD	/api/v1/projects/{project}	Api\ProjectController@show
PUT / PATCH	/api/v1/projects/{project}	Api\ProjectController@update
GET / HEAD	/api/v1/projects/{project}/configuration	Api\ ProjectConfigurationController@show
GET / HEAD	/api/v1/projects/{project}/expenses	Api\ProjectExpenseController@index
POST	/api/v1/projects/{project}/expenses	Api\ProjectExpenseController@store
POST	/api/v1/projects/{project}/finalize	Api\ProjectWorkflowController@finalize
GET / HEAD	/api/v1/projects/{project}/financial-result	Api\FinancialReportController@show
POST	/api/v1/projects/{project}/invitations	Api\ProjectInvitationController@store
POST	/api/v1/projects/{project}/members	Api\ ProjectConfigurationController@addMember
DELETE	/api/v1/projects/{project}/members/{user}	Api\ ManagementController@removeMember
POST	/api/v1/projects/{project}/rates	Api\ ProjectConfigurationController@storeRate
POST	/api/v1/projects/{project}/reopen	Api\ProjectWorkflowController@reopen
PUT	/api/v1/projects/{project}/taxes	Api\ ProjectConfigurationController@replaceTaxes
GET / HEAD	/api/v1/projects/{project}/work-logs	Api\WorkLogController@index
POST	/api/v1/projects/{project}/work-logs	Api\WorkLogController@store
PUT	/api/v1/rates/{rate}	Api\ ManagementController@updateRate
DELETE	/api/v1/rates/{rate}	Api\ ManagementController@deleteRate
GET / HEAD	/api/v1/reports/portfolio	Api\ FinancialReportController@portfolio
GET / HEAD	/api/v1/reports/project-analytics	Api\ProjectAnalyticsController
GET / HEAD	/api/v1/security	Api\AccountSecurityController@show
PUT	/api/v1/security/password	Api\ AccountSecurityController@updatePassword
PUT	/api/v1/security/preferences	Api\ AccountSecurityController@updatePreferences

DELETE	/api/v1/security/providers/{provider}	Api\AccountSecurityController@disconnectProvider
DELETE	/api/v1/security/sessions	Api\AccountSecurityController@revokeOtherSessions
DELETE	/api/v1/security/sessions/{session}	Api\AccountSecurityController@revokeSession
DELETE	/api/v1/security/two-factor	Api\AccountSecurityController@disableTwoFactor
POST	/api/v1/security/two-factor/confirm	Api\AccountSecurityController@confirmTwoFactor
POST	/api/v1/security/two-factor/setup	Api\AccountSecurityController@beginTwoFactor
GET / HEAD	/api/v1/tasks	Api\TaskV1Controller@index
POST	/api/v1/tasks	Api\TaskV1Controller@store
PUT / PATCH	/api/v1/tasks/{task}	Api\TaskV1Controller@update
DELETE	/api/v1/tasks/{task}	Api\TaskV1Controller@destroy
GET / HEAD	/api/v1/teams	Api\TeamController@index
GET / HEAD	/api/v1/teams/{project}	Api\TeamController@show
PUT	/api/v1/teams/{project}	Api\TeamController@update
GET / HEAD	/auth/google	Auth\GoogleAuthController@redirect
GET / HEAD	/auth/google/callback	Auth\GoogleAuthController@callback
GET / HEAD	/auth/microsoft	Auth\MicrosoftAuthController@redirect
GET / HEAD	/auth/microsoft/callback	Auth\MicrosoftAuthController@callback
POST	/forgot-password	Auth\PasswordResetLinkController@store
POST	/logout	Auth\SessionController@destroy
PATCH	/profile	ProfileController@update
POST	/profile/avatar	ProfileAvatarController@update
DELETE	/profile/avatar	ProfileAvatarController@destroy
POST	/reset-password	Auth\NewPasswordController@store
GET / HEAD	/storage/{path}	Closure
PUT	/storage/{path}	Closure

Anexo B. Conferência numérica de cada figura

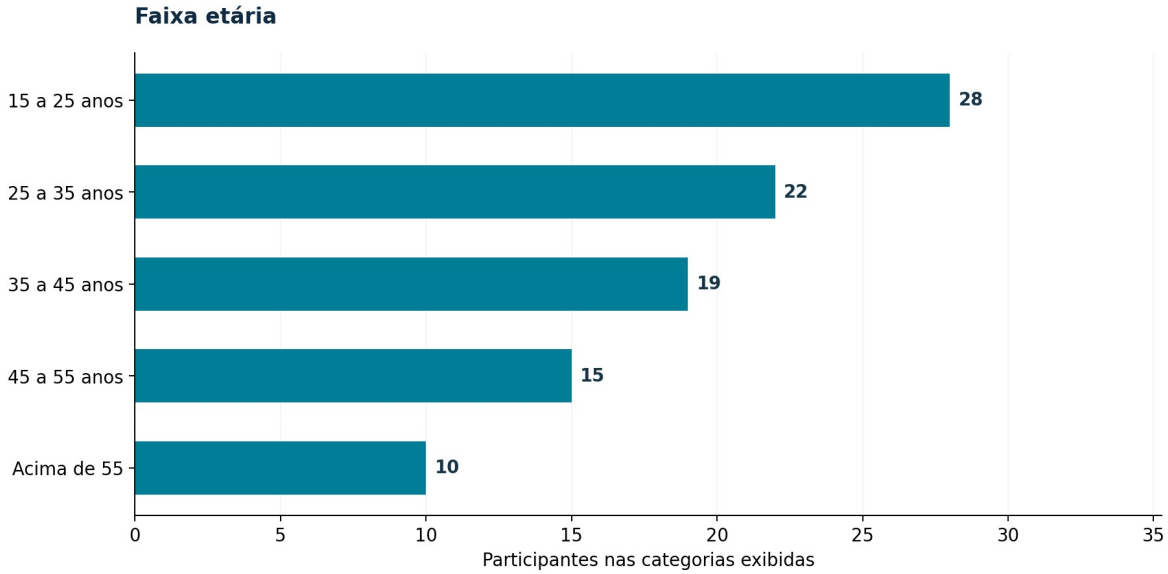
Figura	Tema	Soma exibida	Diferença até 94

1	Faixa etária	94	0
2	Atuação em gestão de projetos	92	2
3	Ferramentas utilizadas	134	Múltiplas menções
4	Software de projetos na empresa	92	2
5	Satisfação com a solução atual	89	5
6	Interface atual é intuitiva?	90	4
7	Fluxo de projetos externos	90	4
8	Fluxo de projetos internos	89	5
9	Dificuldade de organização	89	5
10	Dificuldade de alinhamento	90	4
11	Organização ajuda no faturamento?	91	3
12	Dificuldade de registrar custos extras	89	5
13	Necessidade de software dedicado	90	4

Anexo C. Gráficos completos da pesquisa

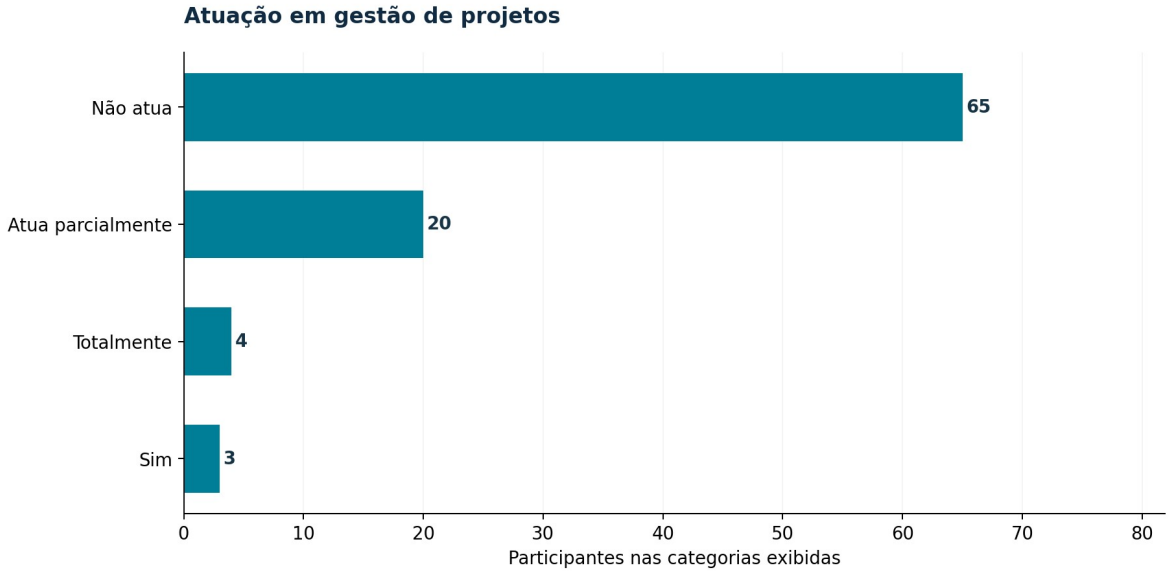
Todos os gráficos abaixo foram reconstruídos dos valores publicados. Nas barras, o valor é contagem, não percentual. A diferença até a base declarada não recebeu categoria inventada.

Figura 1 — Faixa etária



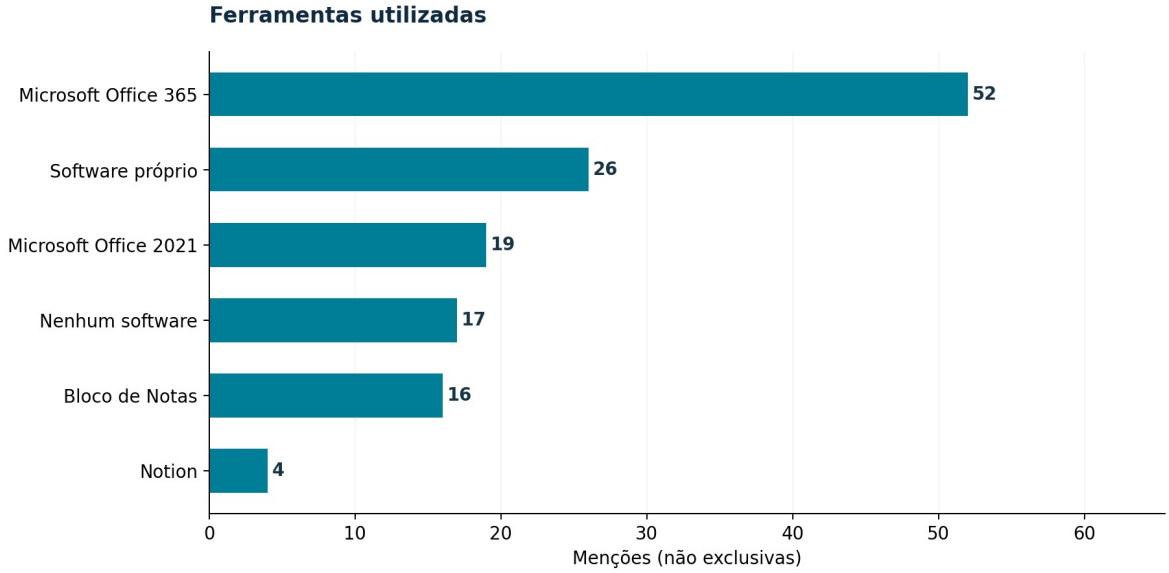
Fonte: pesquisa SPOT (2026), figura 1. Base declarada: 94. Soma exibida: 94; diferença não discriminada: 0.

Figura 2 — Atuação em gestão de projetos



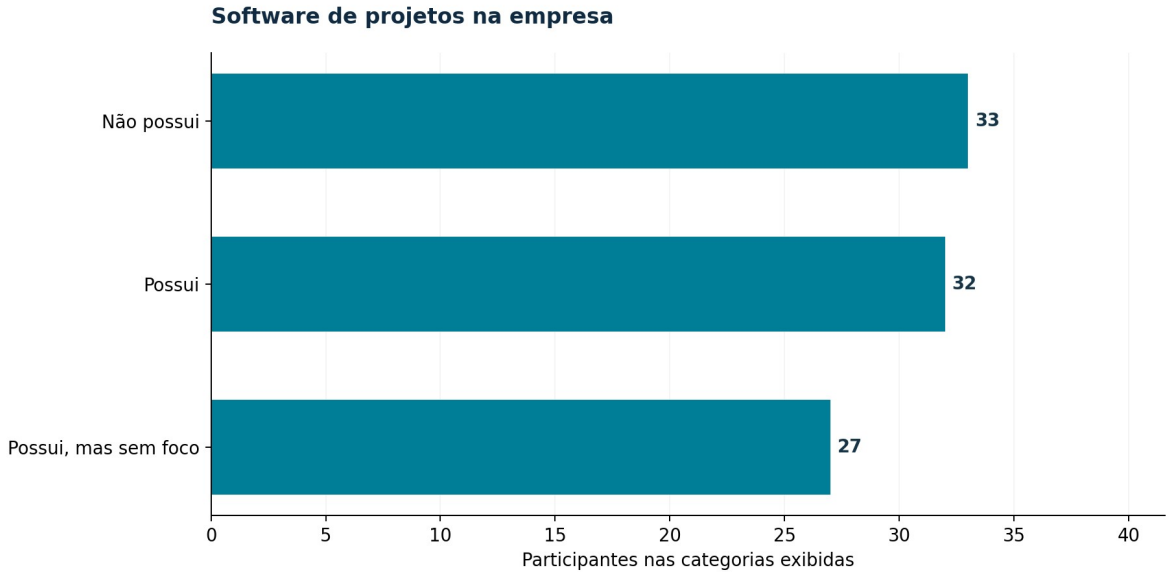
Fonte: pesquisa SPOT (2026), figura 2. Base declarada: 94. Soma exibida: 92; diferença não discriminada: 2.

Figura 3 — Ferramentas utilizadas



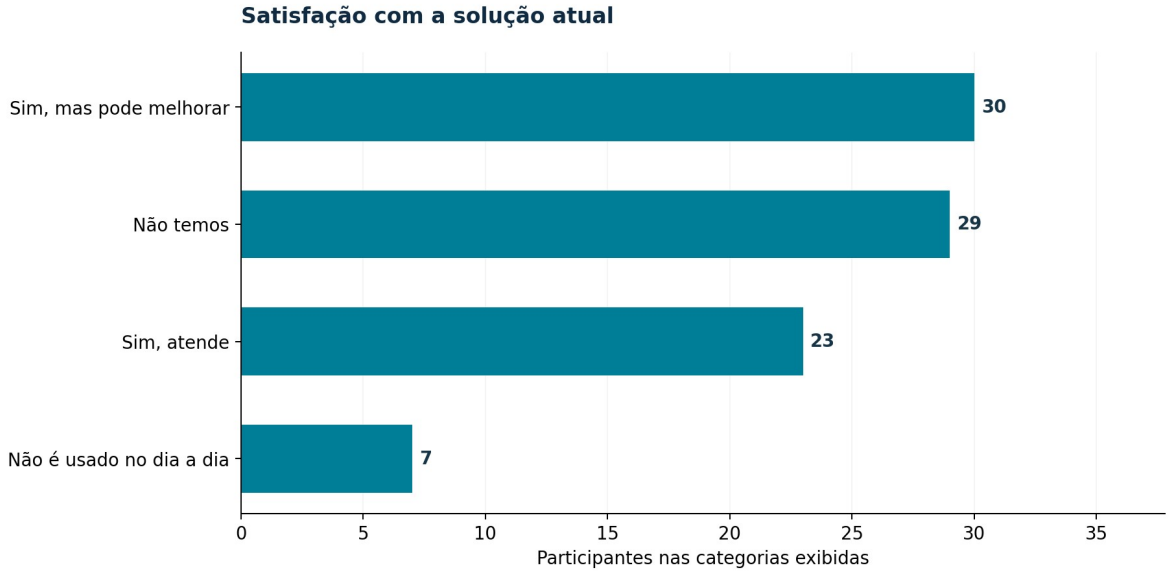
Fonte: pesquisa SPOT (2026), figura 3. Base declarada: 94. Múltiplas menções; não somar percentuais.

Figura 4 — Software de projetos na empresa



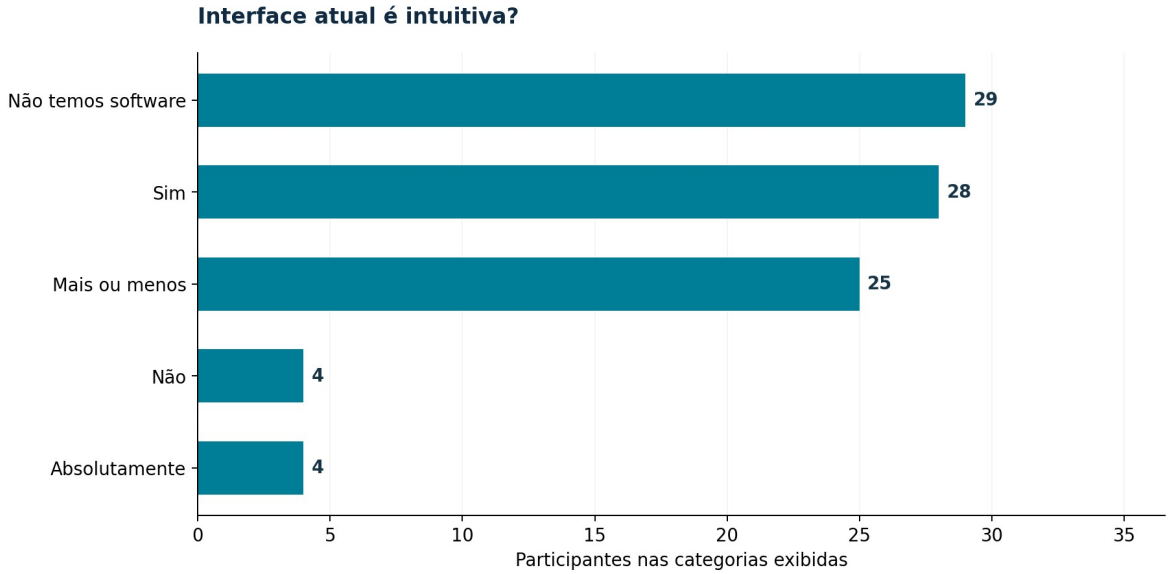
Fonte: pesquisa SPOT (2026), figura 4. Base declarada: 94. Soma exibida: 92; diferença não discriminada: 2.

Figura 5 — Satisfação com a solução atual



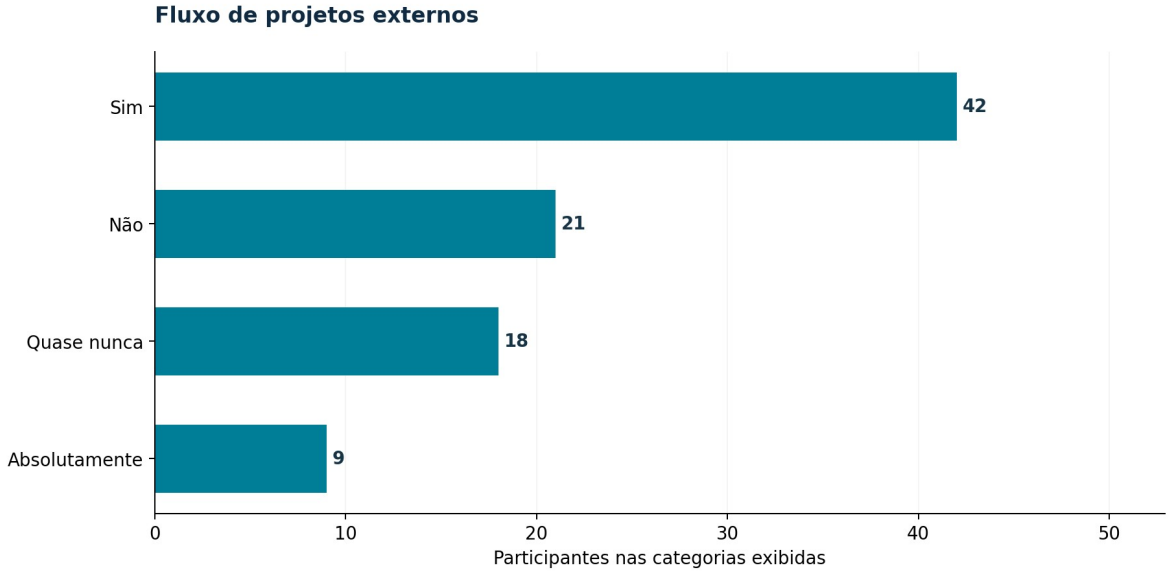
Fonte: pesquisa SPOT (2026), figura 5. Base declarada: 94. Soma exibida: 89; diferença não discriminada: 5.

Figura 6 — Interface atual é intuitiva?



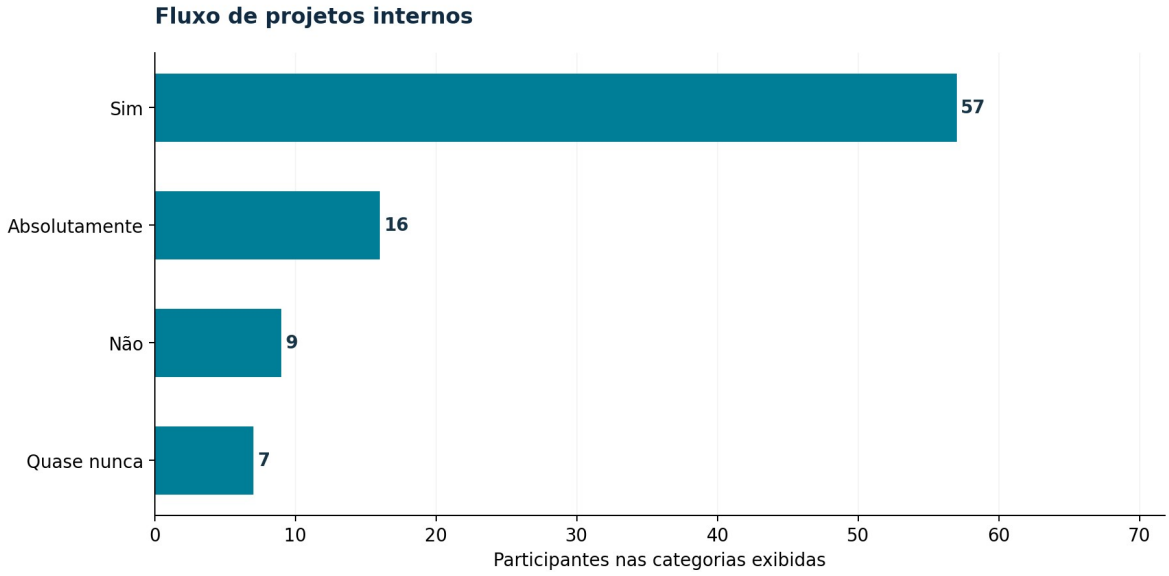
Fonte: pesquisa SPOT (2026), figura 6. Base declarada: 94. Soma exibida: 90; diferença não discriminada: 4.

Figura 7 — Fluxo de projetos externos



Fonte: pesquisa SPOT (2026), figura 7. Base declarada: 94. Soma exibida: 90; diferença não discriminada: 4.

Figura 8 — Fluxo de projetos internos



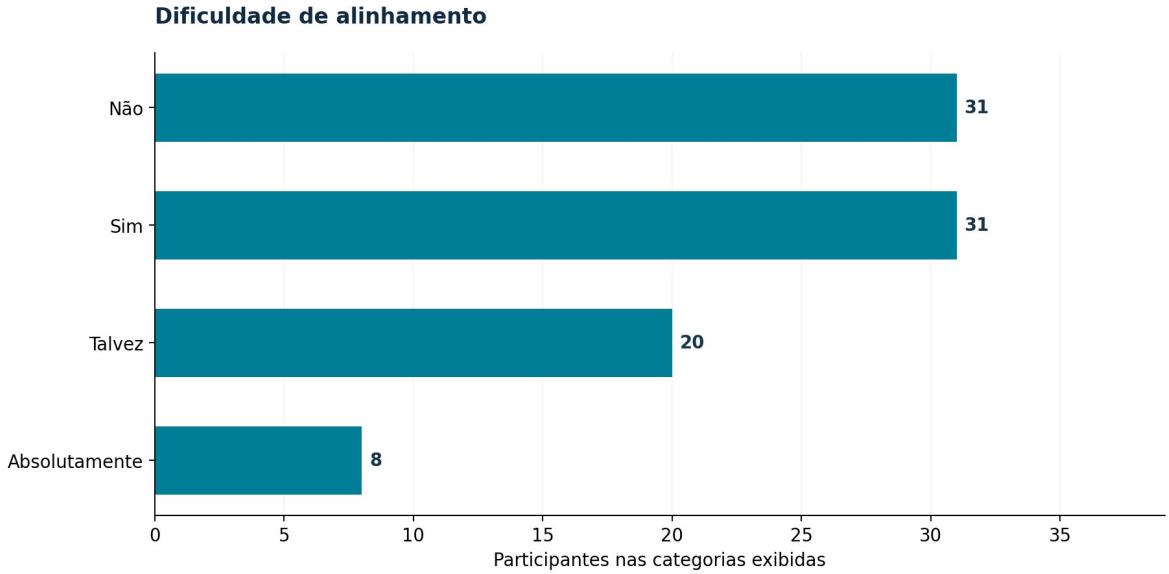
Fonte: pesquisa SPOT (2026), figura 8. Base declarada: 94. Soma exibida: 89; diferença não discriminada: 5.

Figura 9 — Dificuldade de organização



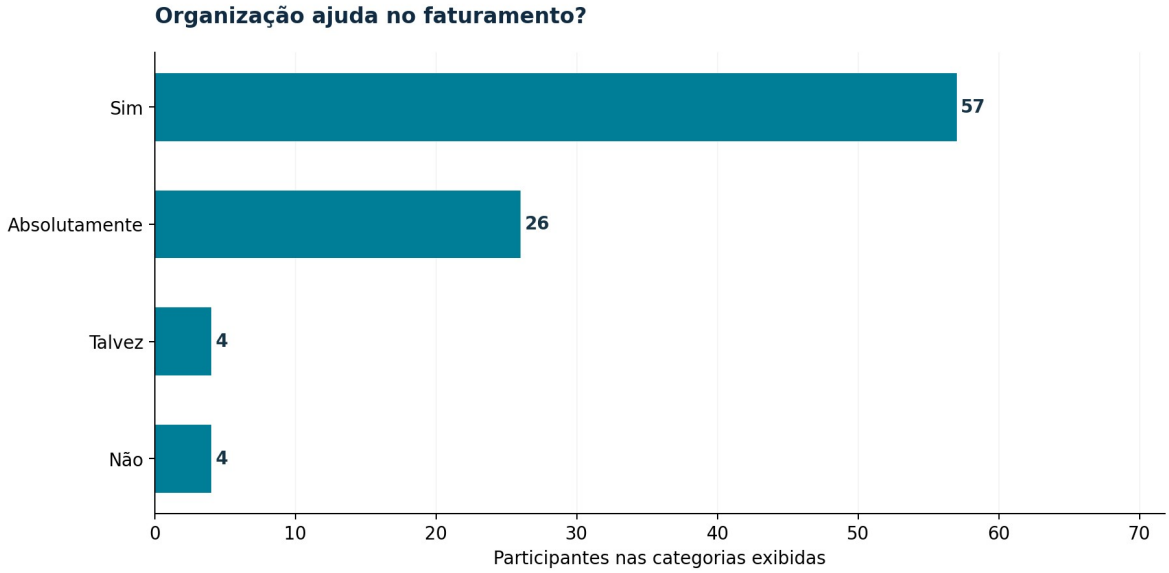
Fonte: pesquisa SPOT (2026), figura 9. Base declarada: 94. Soma exibida: 89; diferença não discriminada: 5.

Figura 10 — Dificuldade de alinhamento



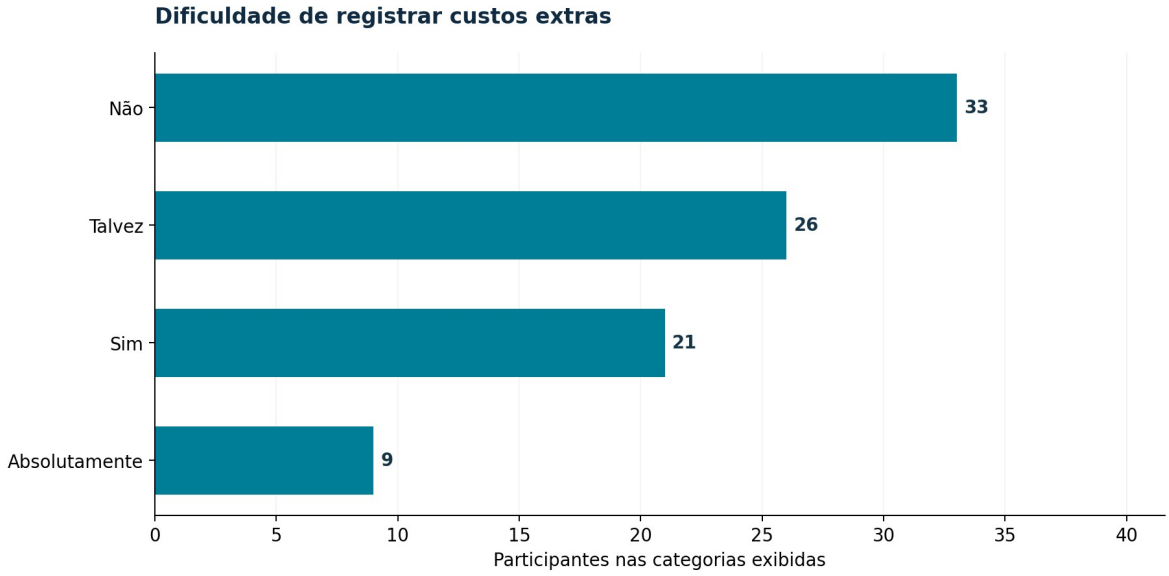
Fonte: pesquisa SPOT (2026), figura 10. Base declarada: 94. Soma exibida: 90; diferença não discriminada: 4.

Figura 11 — Organização ajuda no faturamento?



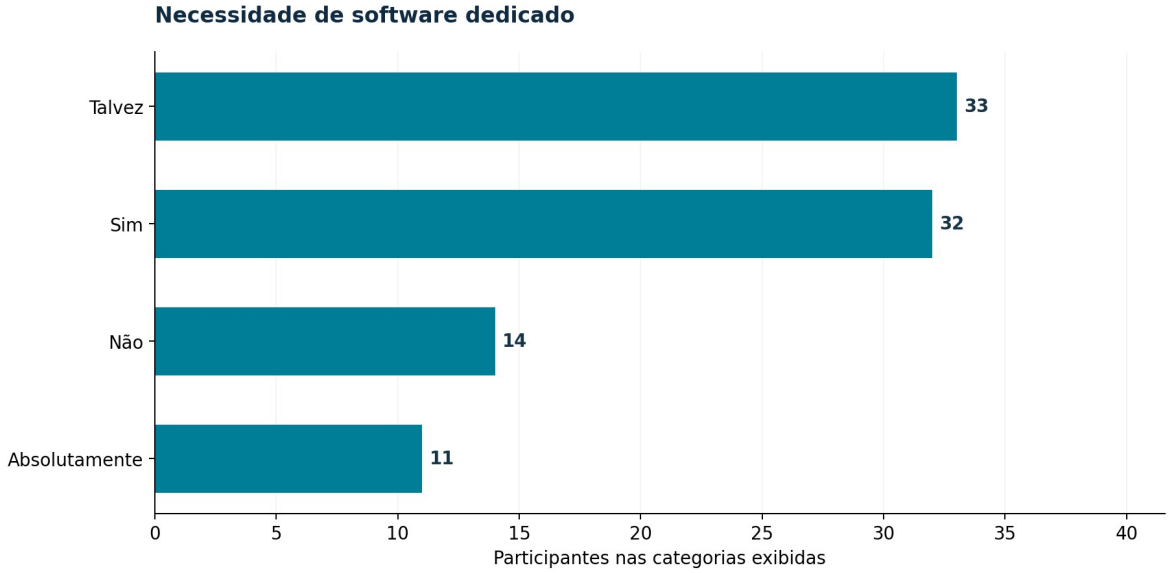
Fonte: pesquisa SPOT (2026), figura 11. Base declarada: 94. Soma exibida: 91; diferença não discriminada: 3.

Figura 12 — Dificuldade de registrar custos extras



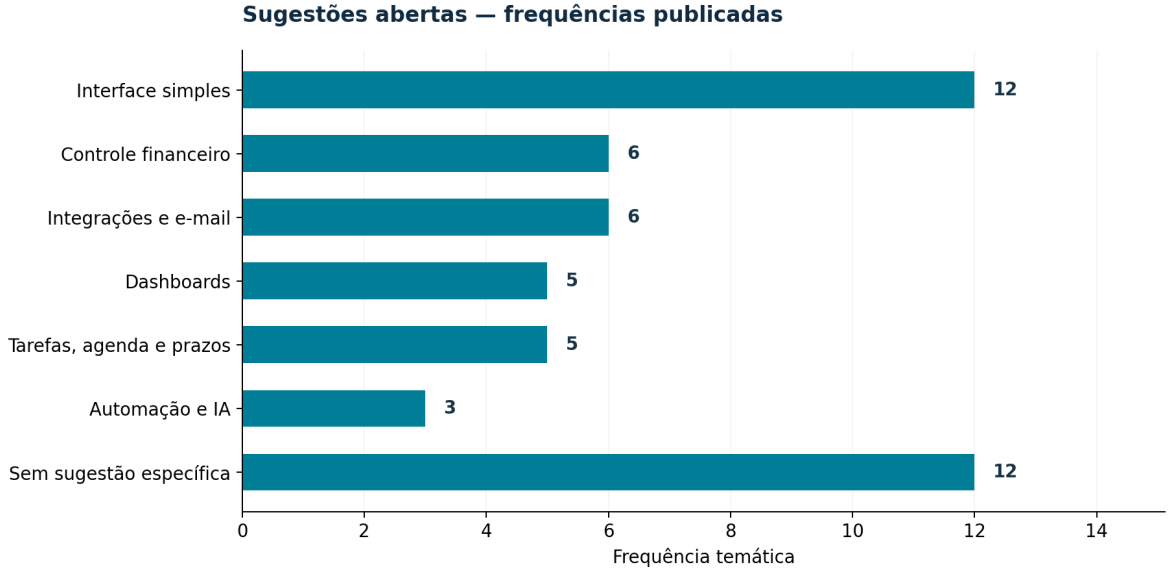
Fonte: pesquisa SPOT (2026), figura 12. Base declarada: 94. Soma exibida: 89; diferença não discriminada: 5.

Figura 13 — Necessidade de software dedicado



Fonte: pesquisa SPOT (2026), figura 13. Base declarada: 94. Soma exibida: 90; diferença não discriminada: 4.

Tabela temática representada em gráfico



Fonte: pesquisa SPOT (2026), tabela 1. Tabela = 49 frequências; texto = 57 comentários.